

It's Time for Time Series

Two Part Lunch and Learn:

July 14th – Time Series Analysis in SAS Studio: Getting Started

August 13th – Time Series Analysis in SAS Studio and Visual Forecasting

Danny Modlin

Principal Analytical Training Consultant, Cary NC

Contents

Abstract	2
Time Series in SAS Studio	3
<i>Time Series Creation</i>	3
<i>Time Series Identification</i>	12
<i>Autocorrelation</i>	16
<i>Exponential Smoothing Model</i>	21
<i>Determining Autoregressive Order (ARIMA Models)</i>	30
<i>Estimation of an ARIMAX Model</i>	33
<i>Using Input Variables in ARIMAX Models</i>	40
<i>Estimation of Input Variables in ARIMAX Models</i>	47
<i>Unobserved Components Model</i>	57
<i>Build a UCM and explore the results</i>	58
Time Series in SAS Viya Model Studio	64
<i>Data Dictionary</i>	64
<i>Setting up a Visual Forecasting Project</i>	65
<i>Creating Pipelines</i>	67
<i>Hierarchical Modeling Node</i>	69

Abstract

Tuesday, July 14th Presentation:

Time Series Analysis in SAS Studio – Getting Started

This presentation will start with an introduction to time series analysis using SAS Studio. Focus will be placed on using both code aspects as well as any available SAS Studio Steps. Topics included will start with time series creation, identification and autocorrelation. We will start creating models such as exponential smoothing, ARIMAX, and unobserved components models.

Thursday, August 13th Presentation:

Time Series Analysis in SAS Studio and Visual Forecasting

In this follow-up presentation, we will finish our discussion of the creation of models such as exponential smoothing, ARIMAX, and unobserved components models. We will then explore the Visual Forecasting interface within SAS Model Studio. We will discuss setting up a project and the creation of models within this interface. A comparison of the SAS Studio and the SAS Model Studio interfaces will be made.

Time Series in SAS Studio

Time Series Creation

This demonstration provides examples of creating time series through the process of accumulation. The Time Series Exploration task in SAS Studio is used to illustrate this. The transactional data set for the analysis is **STSM.CH1_DEMODAT**. The dependent variable is **units** of sales. There are two timestamps: **date** is a date variable and **dtdate** is a datetime variable. A portion of the data is shown below. The transactional observations begin on 02January2004 and end on 29December2006 inclusive.

Obs	units	dtdate	date	mon	yr
1	.	02JAN2004:08:00:00	02JAN2004	1	2004
2	1940	02JAN2004:09:00:00	02JAN2004	1	2004
3	.	02JAN2004:10:00:00	02JAN2004	1	2004
4	3147	02JAN2004:11:00:00	02JAN2004	1	2004
5	.	02JAN2004:12:00:00	02JAN2004	1	2004
6	.	02JAN2004:13:00:00	02JAN2004	1	2004
7	.	02JAN2004:14:00:00	02JAN2004	1	2004
8	.	02JAN2004:15:00:00	02JAN2004	1	2004
9	.	02JAN2004:16:00:00	02JAN2004	1	2004
10	.	02JAN2004:17:00:00	02JAN2004	1	2004

Examining the Data on an Hourly Interval

1. Assign **units** as the dependent variable. Expand the **ADDITIONAL ROLES** property, and assign the **dtdate** column as the time ID.
2. Select **Sum** for the **Accumulation** field.

The screenshot shows the SAS Studio interface with the 'Time Series Exploration' task selected in the left-hand navigation pane. The main workspace displays the configuration for the task, which is named '*Time Series Exploration'.

DATA section:

- DATA: STSM.CH1_DEMODAT
- Filter: (none)

ROLES section:

- Dependent variable: (1 item)
 - units
- Independent variables:
 - Column

Transformations section:

Variable	Accumulation	Transformation
units	Sum	None

ADDITIONAL ROLES section:

- Time ID: (1 item)
 - dtdate

3. After the time ID variable is set, a one-hour interval that begins in the first hour of a 24-hour day is detected as the natural interval of the data. (The options shown below can be changed to modify the detected interval.)

The screenshot shows the 'Time ID: (1 item)' dialog box with the 'dtdate' variable selected. The 'Properties' section is expanded, showing the following settings:

- Interval: Hour
- Multiplier: 1
- Shift: 1
- Season length: 24

Note: The TIMEID procedure in SAS/ETS runs “under the hood” in SAS Studio to detect the interval of the data.

4. Click  on the toolbar to maximize the view.

The TIMESERIES procedure in SAS/ETS provides the main functionality for the Time Series Exploration task. A subset of the contents of the CODE window is shown below.

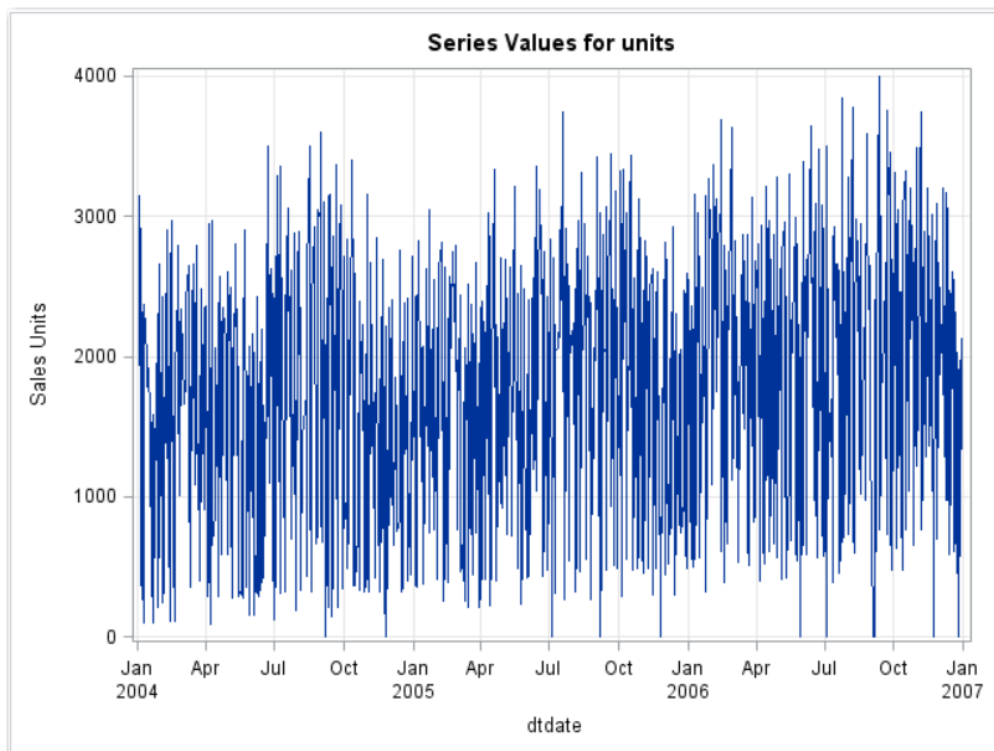
```
proc sort data=STSM.CH1_DEMODAT out=Work.preProcessedData;  
  by dtdate;  
run;  
  
proc timeseries data=Work.preProcessedData seasonality=24  
  plots=(series corr);  
  id dtdate interval=hour;  
  var units / accumulate=total transform=none dif=0 sdif=0;  
run;
```

- The SORT procedure is used to create a working copy of the transactional data and sort it from the earliest to the latest date. In different versions of SAS Studio, the naming convention of the sorted data might be different.
- The TIMESERIES procedure is used to accumulate the transactional data to an hourly interval. The PLOTS option in the procedure statement creates the series plot and correlation panel. The ID statement defines the datetime ID variable and the interval and accumulation options that correspond to the settings that are selected on the DATA tab.

Note: If you produce the code by entering it directly in the editor, the code below produces the necessary output. This code assumes that the data in **stsm.ch1_demodat** are properly sorted.

```
proc timeseries data=stsm.ch1_demodat out=out_totalhour  
  plots=(series);  
  id dtdate interval=dthour accumulate=total;  
  var units;  
run;
```

- To submit the code and run the Time Series Exploration task, click **Run** on the toolbar. The series plot is shown below. There are many hourly intervals in a three-year time span.



- To get a better look at the hourly, intervalled time series, the data is truncated and then re-plotted. Because there is currently no option to do this with the DATA properties of SAS Studio, the PROC TIMESERIES syntax is edited directly. Click the **CODE** tab and then click **Edit**.



Note: The syntax is now contained on a new tab.

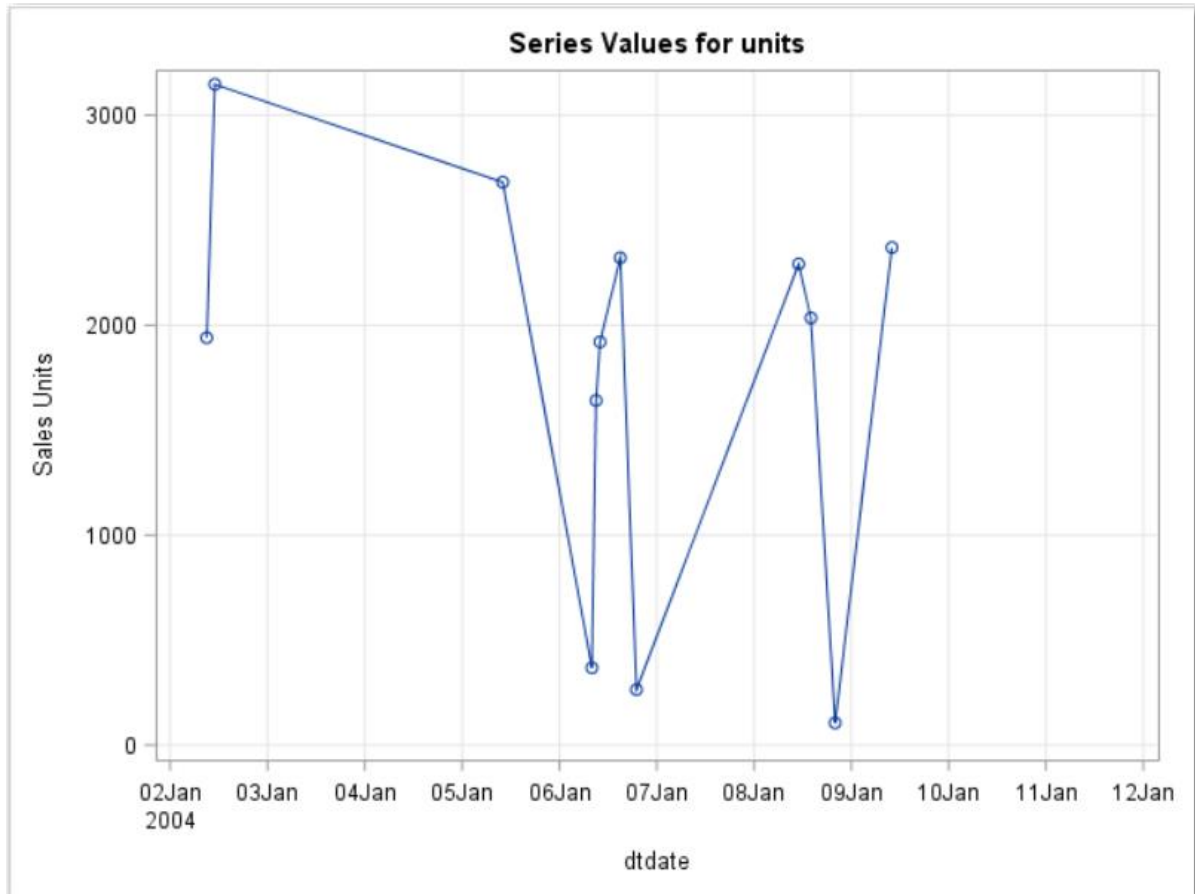
- Add an END option to the ID statement in PROC TIMESERIES as shown below. The end value `12jan2004:00:00` truncates the data so that only the first 10 days of data are shown in the plots.

```
proc timeseries data=Work.preProcessedData seasonality=24
                plots=(series corr) ;
    id dtdate interval=hour end='12jan2004:00:00'dt;
    var units / accumulate=total transform=none dif=0 sdif=0;
run;
```

Note: If you produce the code by entering it directly in the editor, the following code produces the necessary output:

```
proc timeseries data=out_totalhour plots=(series);  
  id dtdate interval=dthour accumulate=total  
    end='12jan2004:00:00'dt;  
  var units;  
run;
```

8. Click **Run** to submit the modified syntax.



One important point, illustrated in the plot above and the table below, is that issues with missing observations cannot be discerned until after the data is accumulated. If an hourly or daily interval is used for forecasting, a data imputation method might be necessary.

9. Click the **OUTPUT DATA** tab to view the hourly, intervalled, time series data.

	dtdate	units
1	02JAN04:08	.
2	02JAN04:09	1940
3	02JAN04:10	.
4	02JAN04:11	3147
5	02JAN04:12	.
6	02JAN04:13	.
7	02JAN04:14	.
8	02JAN04:15	.
9	02JAN04:16	.
10	02JAN04:17	.
11	02JAN04:18	.

Note: The SETMISSING option in the ID statement of the TIMESERIES procedure can be used to replace missing values in time series with imputed values.

Note: A preferred method for missing value imputation in time series is replacing missing values with their one-step-ahead forecast. The ESM procedure in SAS/ETS can be used for this. See the REPLACEMISSING option in the FORECAST statement.

Creating the Time Series on a Monthly Interval

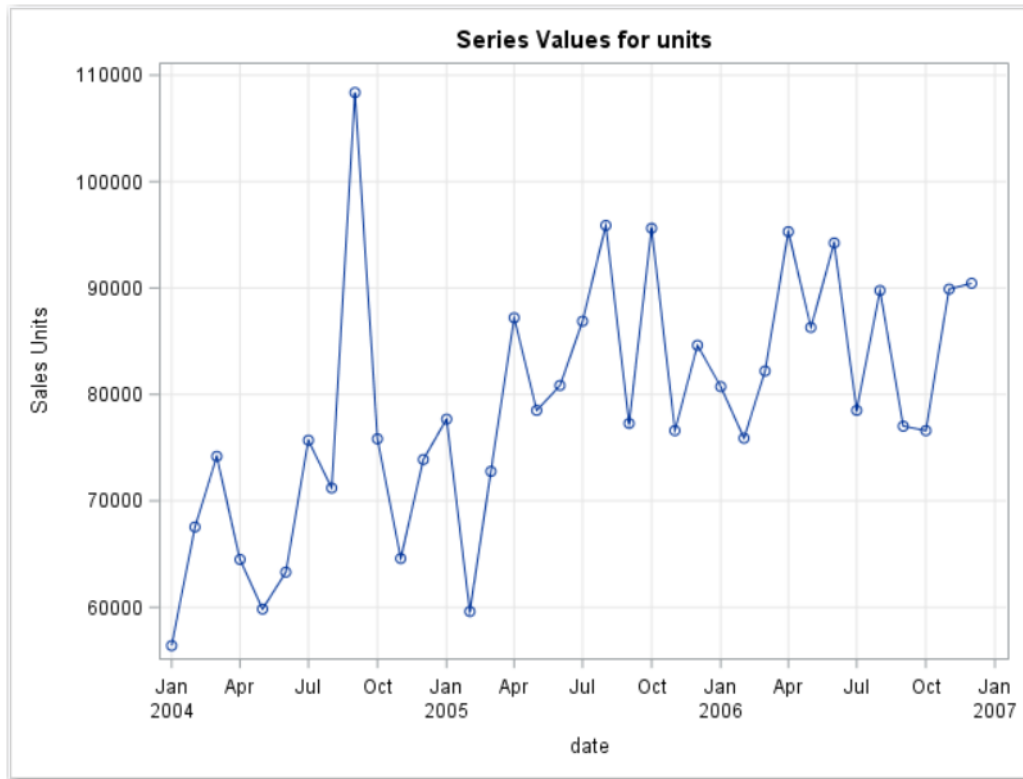
1. Click the **Time Series Exploration 1** tab to return to the original analysis.
 2. Remove the assignment of the time ID variable. Highlight **dtdate** and click the **Trash Can** icon.
 3. Assign **date** as the time ID variable and change the interval property to **Month**.
 4. Expand the **Transformations** properties and set the **Accumulation** field to **Sum**.
- These steps are summarized in the display below.

DATA	ANALYSES	INFORMATION						
▲ ROLES * Dependent variable: <i>(1 item)</i> units + Independent variables: Column +								
▲ Transformations <table border="1" style="width: 100%; border-collapse: collapse; margin-top: 10px;"> <thead> <tr style="background-color: #f0f0f0;"> <th style="text-align: left; padding: 5px;">Variable</th> <th style="text-align: left; padding: 5px;">Accumulation</th> <th style="text-align: left; padding: 5px;">Transformation</th> </tr> </thead> <tbody> <tr> <td style="padding: 5px;">units</td> <td style="padding: 5px;">Sum ▼</td> <td style="padding: 5px;">None ▼</td> </tr> </tbody> </table>			Variable	Accumulation	Transformation	units	Sum ▼	None ▼
Variable	Accumulation	Transformation						
units	Sum ▼	None ▼						
▲ ADDITIONAL ROLES Time ID: <i>(1 item)</i> date +								
▲ Properties Interval: Month ▼ Multiplier: 1 ↑ ↓ Shift: 1 ↑ ↓ Season length: 12 ↑ ↓								

Note: If you produce the code by entering it directly in the editor, the following code produces the necessary output:

```
proc timeseries data=stsm.ch1_demodat out=out_totalmonth
               plots=(series);
  id date interval=month accumulate=total;
  var units;
run;
```

10. Click **Run** to submit the modified Time Series Exploration task. A plot of the monthly, intervalled **units** time series is shown below.

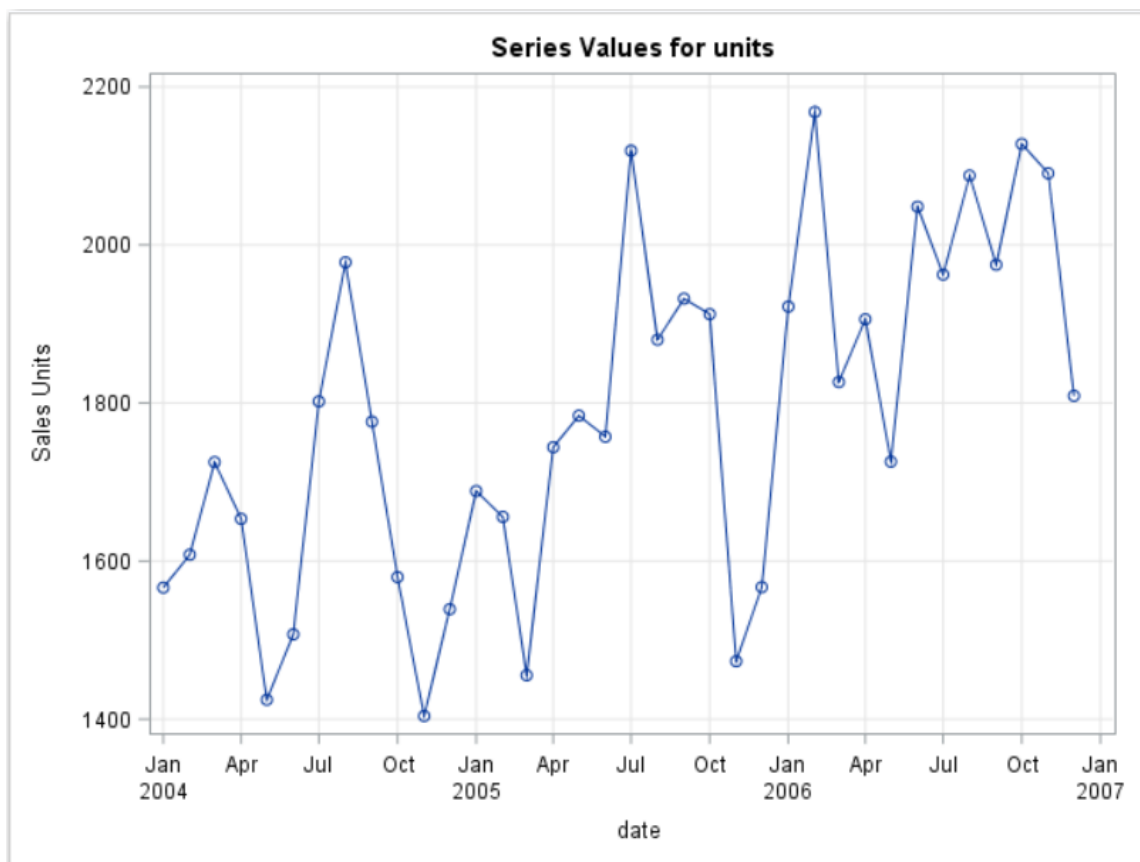


Using a Total or Sum accumulation method, the monthly, intervalled data shows what might be a structural break near the middle of 2005. The spike in Sep2004 might be an anomaly that requires further investigation and consideration when you build a model to accommodate the variation in this series.

11. Change the **Accumulation** field value to **Average**.
12. If you produce the code by entering it directly in the editor, the following code produces the necessary output:

```
proc timeseries data=stsm.ch1_demodat out=out_totalmonth
                plots=(series);
    id date interval=month accumulate=average;
    var units;
run;
```

13. Click **Run**.



The plot of the time series that was created using an Average accumulation method reveals a trend and a cycle in the time series data.

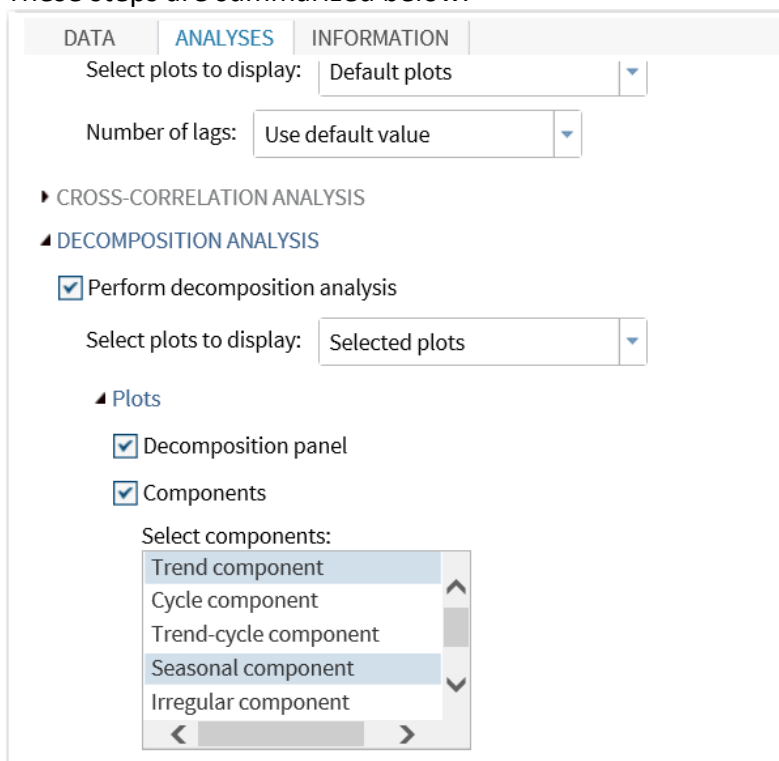
End of Demonstration

Time Series Identification

This demonstration begins where the previous demonstration ended. The monthly, intervalled time series that was accumulated using the Average method is decomposed to assess the presence of trend, seasonal, and other components of systematic variation.

1. Click the **ANALYSES** tab in the Time Series Exploration task that you created in the previous demonstration.
2. Expand the **DECOMPOSITION ANALYSIS** properties.
3. Select the **Perform decomposition analysis** check box and change the **Select plots to display** field to **Selected plots**.
4. Select the **Components** check box and highlight the **Trend** and **Seasonal** component plots.

These steps are summarized below.



The screenshot shows the SAS Time Series Explorer interface with the **ANALYSES** tab selected. The **DECOMPOSITION ANALYSIS** section is expanded, showing the **Perform decomposition analysis** checkbox checked and the **Select plots to display** dropdown set to **Selected plots**. Under the **Plots** section, the **Components** checkbox is checked, and a list of components is shown with **Trend component** and **Seasonal component** highlighted.

5. Use SAS/ETS program code.

Additional syntax includes the following decomposition plot options:

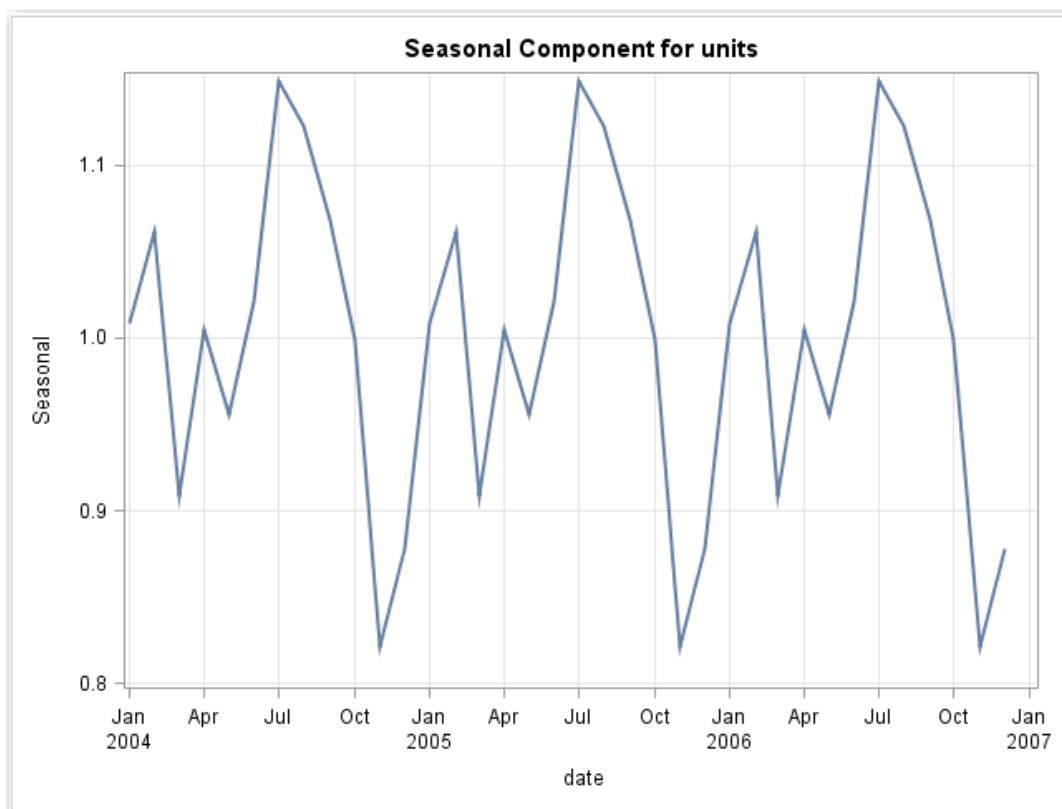
- TC = Trend Component
- SC = Seasonal Component
- IC = Irregular Component
- CORR plots correlation functions

The **decomp_total** data set contains the decomposition statistics and the *de-seasonalized series*. (The estimated seasonal component is subtracted from the original series to create the output series.)

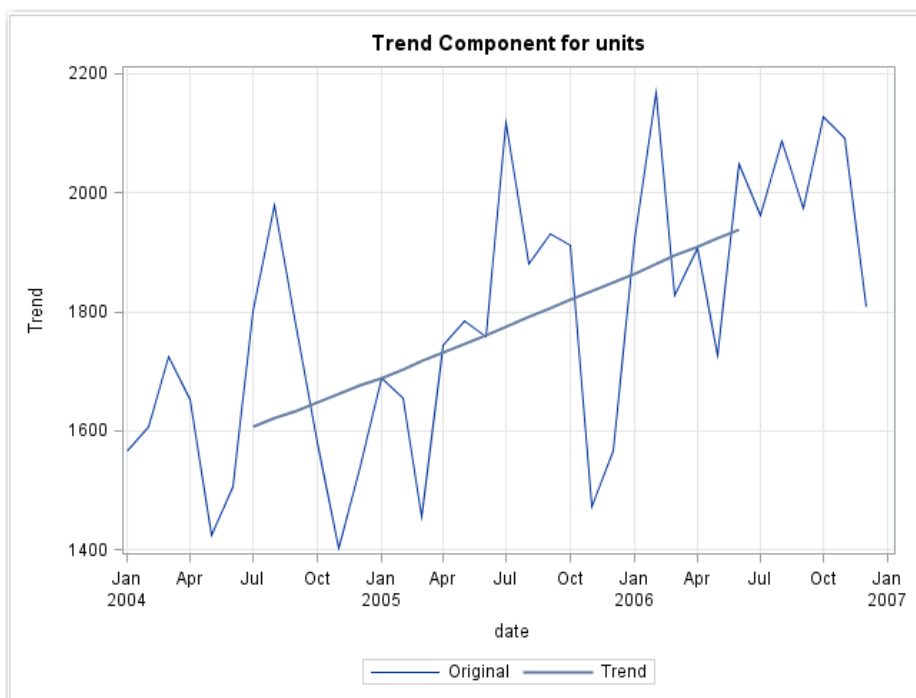
```
proc timeseries data=stsm.ch1_demodat out=out_totalmonth  
    outdecomp=decomp_total  
    print=(descstats seasons decomp)  
    plot=(series corr tc sc ic);  
    id date interval=month accumulate=average;  
    var units;  
run;
```

6. Click **Run**.

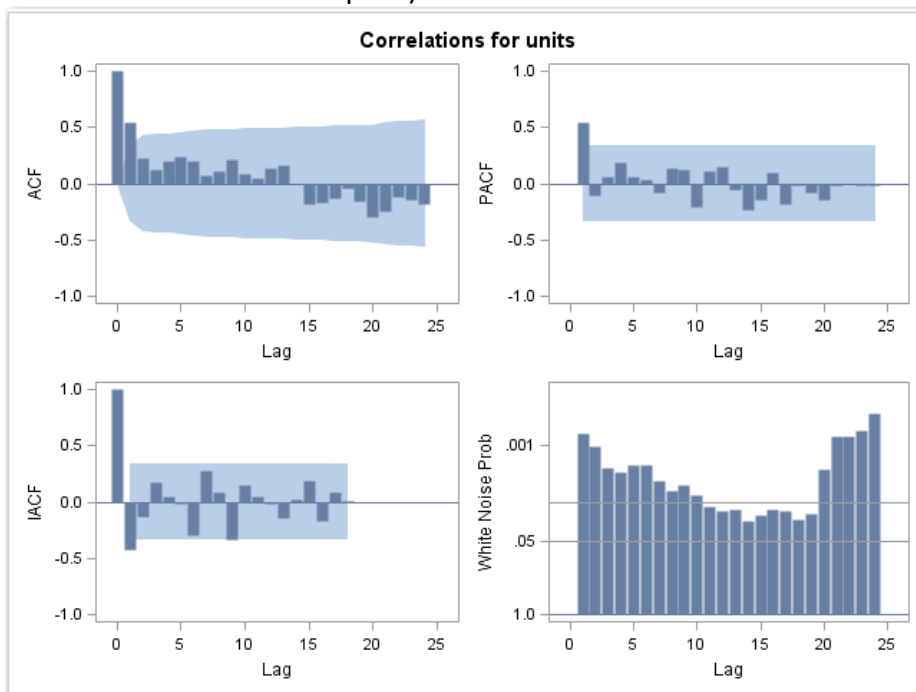
The seasonality plot reveals a strong seasonal cycle in the data that was accumulated to a monthly average. The seasonal peak month is July. Sales of **units** in July average more than 10% above average, annual sales. November is the seasonal low month. Sales in November average are almost 20% below the annual average.



The trend plot is based on a centered, moving-average representation of the series. A strong and linear increase in average sales per month is shown.



The correlation panel shows that the average **units** series is not white noise. The additional diagnostics provide information that can be used to select autoregressive and moving average orders for ARIMA models. (More information about these is provided in the “ARIMAX Models” chapter.)



A portion of the **decomp_total** data set is shown below. There are 12 unique values of the seasonal components. The peak month, July, is approximately 15% above the annual average. The Seasonally Adjusted series is derived by dividing the **units** (Original) series by the Seasonal Component series.

date	Seasonal Index	Original Series	Trend-Cycle Component	Seasonal Component	Irregular Component	Seasonally Adjusted Series
JAN2004	1	1566.44	.	1.00888	.	1552.65
FEB2004	2	1608.07	.	1.06163	.	1514.72
MAR2004	3	1725.09	.	0.90875	.	1898.30
APR2004	4	1653.51	.	1.00518	.	1644.99
MAY2004	5	1424.55	.	0.95608	.	1489.98
JUN2004	6	1507.31	.	1.02105	.	1476.23
JUL2004	7	1802.05	1635.47	1.14892	0.95903	1568.47
AUG2004	8	1978.14	1642.56	1.12292	1.07247	1761.60
SEP2004	9	1776.46	1633.32	1.06845	1.01796	1662.64
OCT2004	10	1579.73	1625.86	0.99908	0.97252	1581.19
NOV2004	11	1404.00	1644.62	0.82108	1.03972	1709.95
DEC2004	12	1539.23	1670.01	0.87796	1.04980	1753.18
JAN2005	1	1688.65	1693.65	1.00888	0.98827	1673.79
FEB2005	2	1655.89	1702.78	1.06163	0.91601	1559.76
MAR2005	3	1455.50	1705.19	0.90875	0.93928	1601.64

End of Demonstration

Autocorrelation

This demonstration uses the **stsm.solarpv** data set and the Time Series Exploration task to help visualize the **kW_Gen** series and determine whether there is a systematic variation that can be used to forecast future periods of solar power generation. By analyzing the autocorrelation function plot and the white noise probability plot, you can determine whether the series is white noise.

The variables in the **stsm.solarpv** data set are the following:

- **EDT**: time interval (weekly)
- **kW_Gen**: kilowatts of solar power generated (averaged per day)
- **Cloud_Cover**: a metric that quantifies average weekly cloud cover in the area

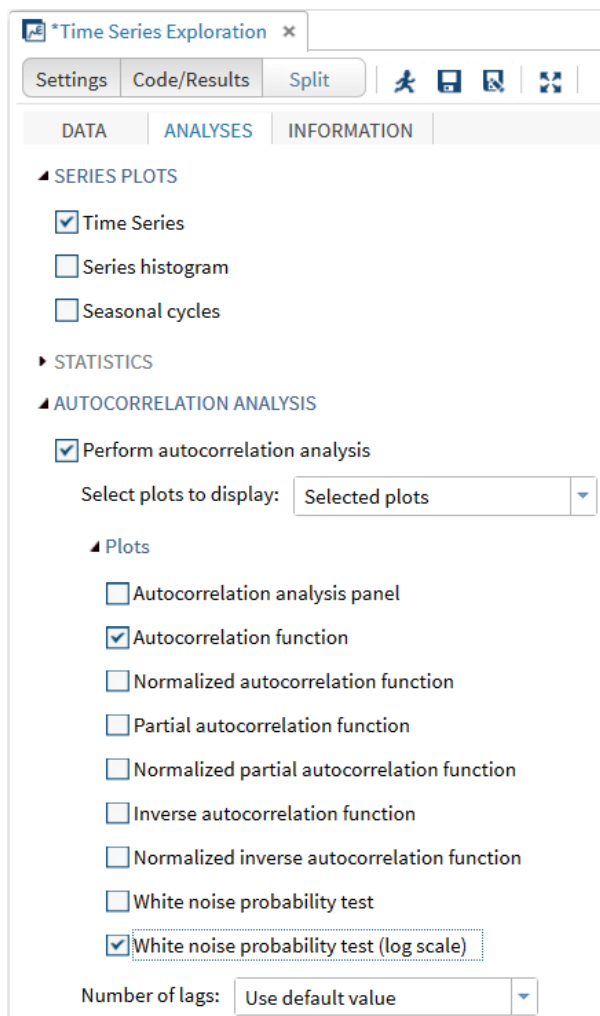
The screenshot shows the SAS Time Series Explorer interface with the following configuration:

- Settings** | **Code/Results** | **Split**
- DATA** | **ANALYSES** | **INFORMATION**
- DATA**
 - STSM.SOLARPV
- ROLES**
 - Dependent variable: (1 item)
 - kW_Gen
 - Independent variables:
 - Column
- Transformations**

Variable	Accumulation	Transformation	Simple Differ...	Seasonal Differ...
kW_Gen	None	None	0	0
- ADDITIONAL ROLES**
 - Time ID: (1 item)
 - EDT
- Properties**
 - Interval: Week
 - Multiplier: 1
 - Shift: 1
 - Season length: 52

Under AUTOCORRELATION ANALYSES on the Split tab, only **Autocorrelation function** and **White noise probability test (log scale)** are selected. A quick visual inspection of both plots determines whether the series is white noise.

Note: Why was **White noise probability test (log scale)** selected instead of *White noise probability test*? The log scale plot conforms to the “White Means white.” phrase from the earlier slide. **Be cautious of the Y axis.** Both plots lead to the same conclusion, but they can be easily misinterpreted if you do not pay careful attention to the Y axis.



The code generated by SAS Studio is as follows:

```
proc sort data=STSM.SOLARPV out=Work.preProcessedData;
    by EDT;
run;

proc timeseries data=Work.preProcessedData seasonality=52
    plots=(series acf wn);
    id EDT interval=week;
    var kW_Gen / accumulate=none transform=none dif=0 sdif=0;
    ods exclude ACFNORMPlot;
    ods exclude WhiteNoiseProbabilityPlot;
run;
```

```
/* Remove the temp data set */
proc delete data=Work.preProcessedData;
run;
```

Note: Alternatively, you can write the code directly in SAS/ETS.

```
proc timeseries data=STSM.solarpv
               seasonality=52
               plots=(series acf wn);
  id EDT interval=week;
  var kW_Gen;
  ods exclude ACFNORMPlot WhiteNoiseProbabilityPlot;
run;
```

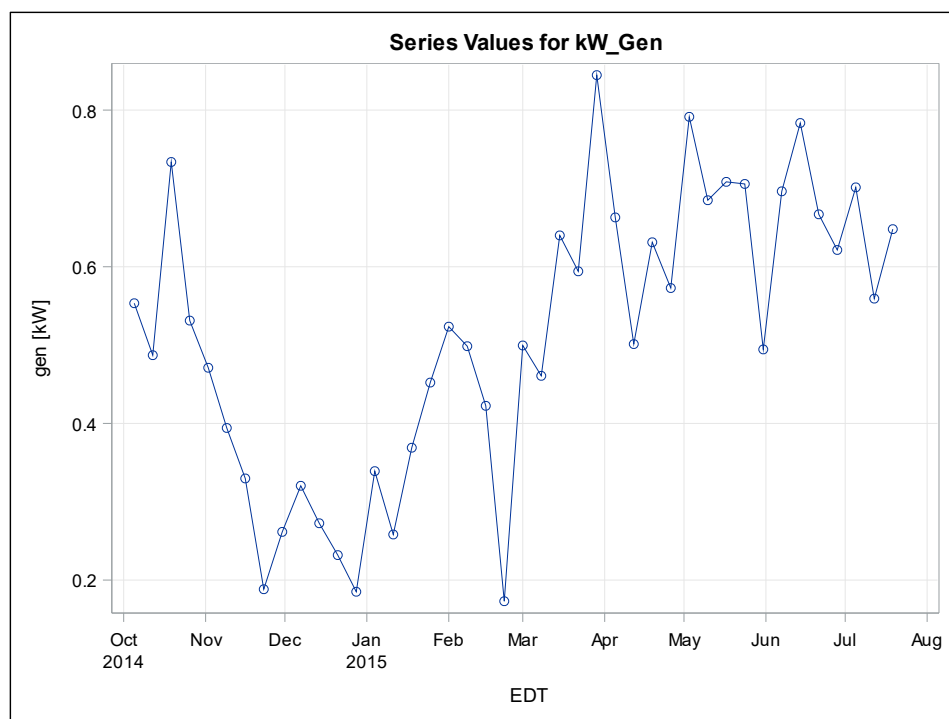
SAS Studio uses a **Work.preProcessedData** data set to do the analysis instead of the original data set **stsm.solarpv**. This is true for other tasks in SAS Studio as well. This method ensures that the original data set remains unchanged throughout the entire process. (In Chapter 2, the displayed code does not include the SORT or DELETE procedures.)

Input Data Set	
Name	WORK.PREPROCESSEDDATA
Label	
Time ID Variable	EDT
Time Interval	WEEK
Length of Seasonal Cycle	52

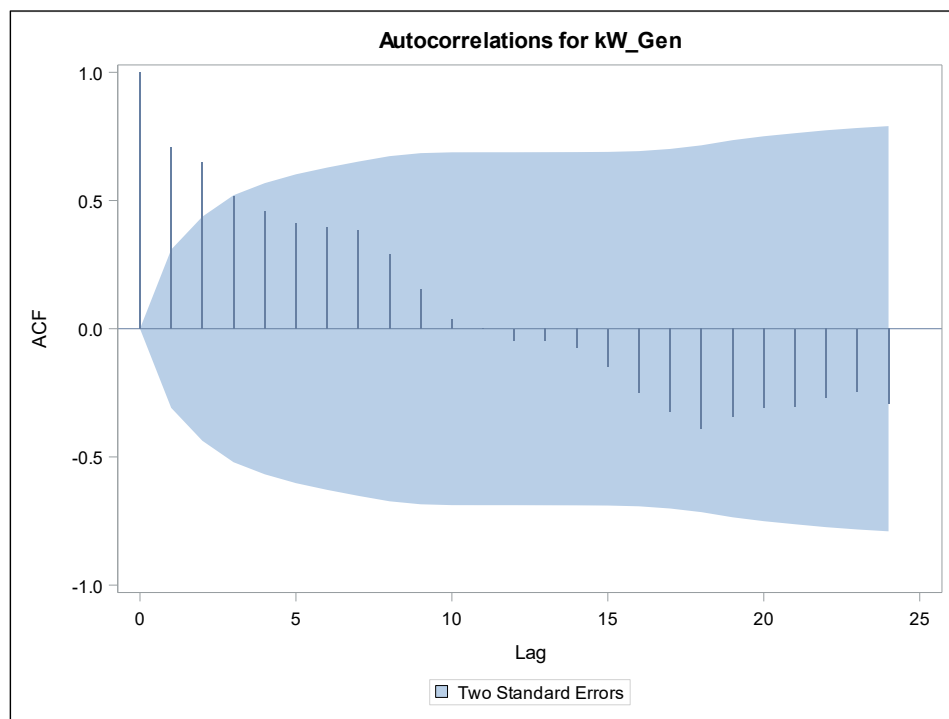
The Variable Information table provides basic information about the series, and enables confirmation that the appropriate data and time range are used.

Variable Information	
Name	kW_Gen
Label	gen [kW]
First	Sun, 5 Oct 2014
Last	Sun, 19 Jul 2015
Number of Observations Read	42

A plot of the data is generated as part of the output. Because no transformation was applied, the plot is of the raw **kW_Gen** series. Because the Time Series Exploration task creates this plot, graphing the series using the Graph task in SAS Studio is unnecessary.

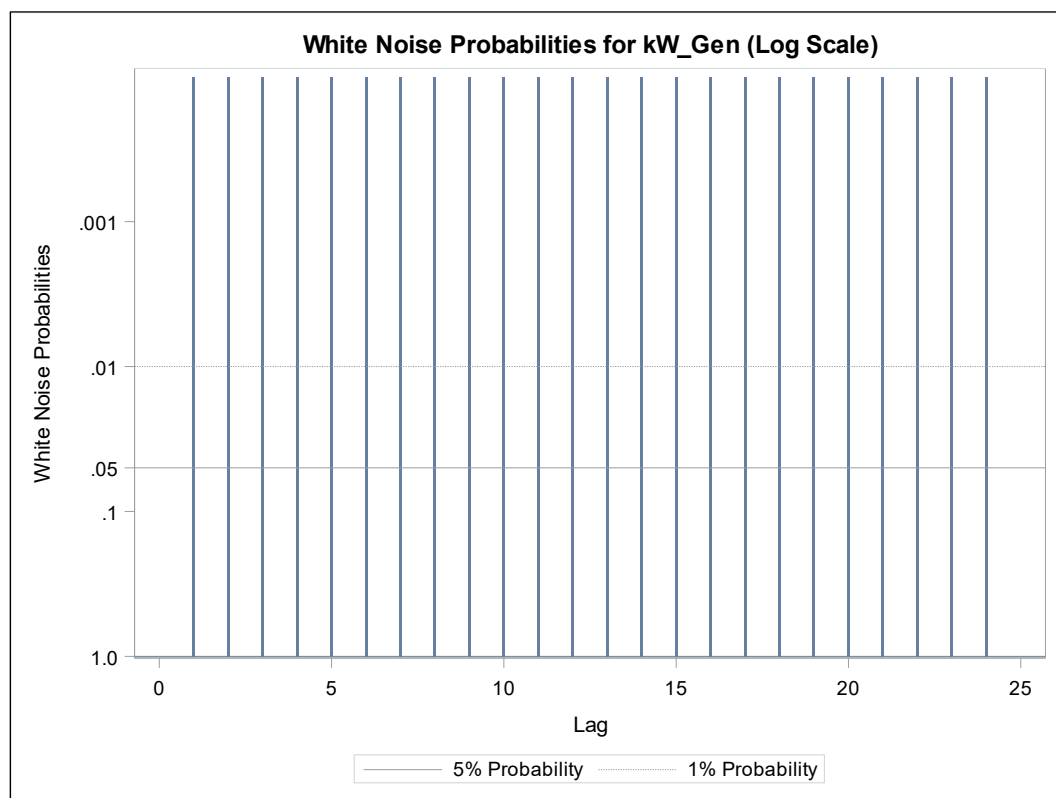


The autocorrelation function plot (ACF) shows at least one nonzero lag with a significant spike. This indicates that the series contains autocorrelation. The 95% confidence intervals are higher due to having only 42 observations, but there are clearly two nonzero lags with significant spikes.



The White Noise Probabilities plot confirms that the series is **not** white noise, and that the series contains systematic variation that can be used to forecast. Recall that the null hypothesis for the White

Noise test is that the series is white noise. The White Noise Probabilities plot strongly rejects the null hypothesis at all lags. The conclusion is that the series is not white noise.



End of Demonstration

Exponential Smoothing Model

In this demonstration, you use the Time Series Exploration task and the Modeling and Forecasting task in SAS Studio to explore the data set and generate an exponential smoothing model for sea surface temperature. The **SST** data set can be found in the **STSM** library. Use the Time Series Exploration task to investigate the possible inclusion of a trend or seasonality component. Then, use the Modeling and Forecasting task to generate the appropriate exponential smoothing models. Include forecasts that are 12 months into the future.

1. Expand the **Tasks** area in the left navigation pane as well as the **Forecasting** subsection. Double-click the **Time Series Exploration** task. Click the **Maximized View** button.
2. On the DATA tab, select the **SST** data set within the **STSM** library.
3. Click the plus sign (+) next to **Dependent variable**. Select **temp**. Click **OK**. For this model, there is no independent variable.
4. Click and expand the **Additional Roles** section. Click the plus sign (+) next to **Time ID**. Select **Datechk**. Click **OK**. Notice that the Properties section is updated to reflect the interval of monthly data.
5. Click the **Analyses** tab. Select the **Seasonal cycles** check box. Do **not** clear the **Time Series** check box. Under Autocorrelation Analysis, make sure that the **Perform autocorrelation analysis** check box is selected. Change the drop-down box beside **Select plots** to display **Selected plots**.
6. Under Plots, select the **Autocorrelation analysis panel** and **White noise probability test (log scale)** check boxes.
7. Under Decomposition Analysis, make sure that the **Perform decomposition analysis** check box is selected. Change the drop-down box beside **Select plots** to display **Selected plots**.
8. Select the four check boxes (**Decomposition panel**, **Components**, **Seasonally adjusted series**, and **Seasonally adjusted series (percent change)**) under Plots.
9. When **Components** is selected, select **Trend component** and **Seasonal component** only.

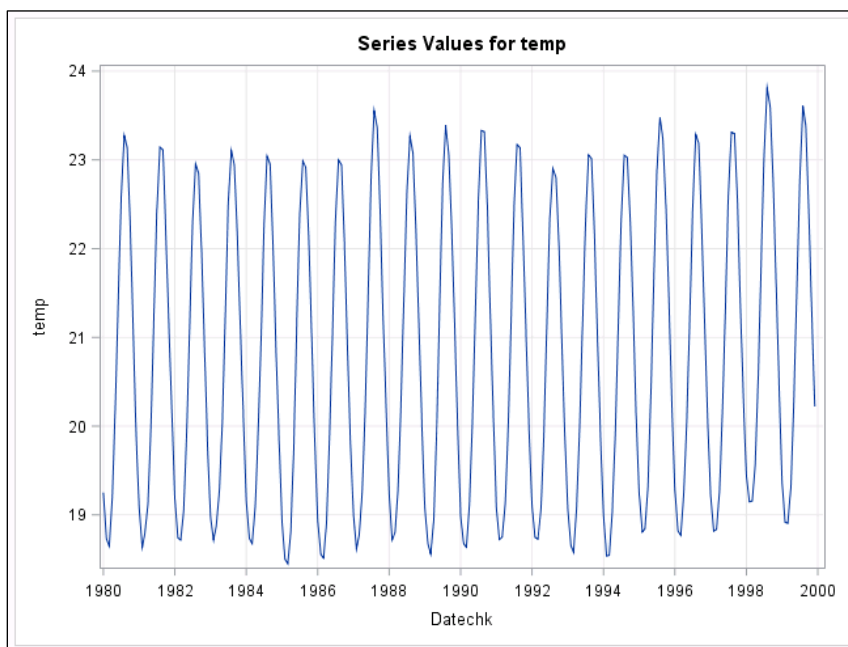
Note: Alternatively, you can write the SAS code directly as follows:

```
proc timeseries data=STSM.SST
               seasonality=12
               plots=(series cycles corr wn
                      decomp sa pcsa tc sc);
  id Datechk interval=month;
  var temp;
  decomp sa pcsa tc sc / mode=multoradd;
  ods exclude WhiteNoiseProbabilityPlot;
run;
```

10. Click the running person icon to execute the task.

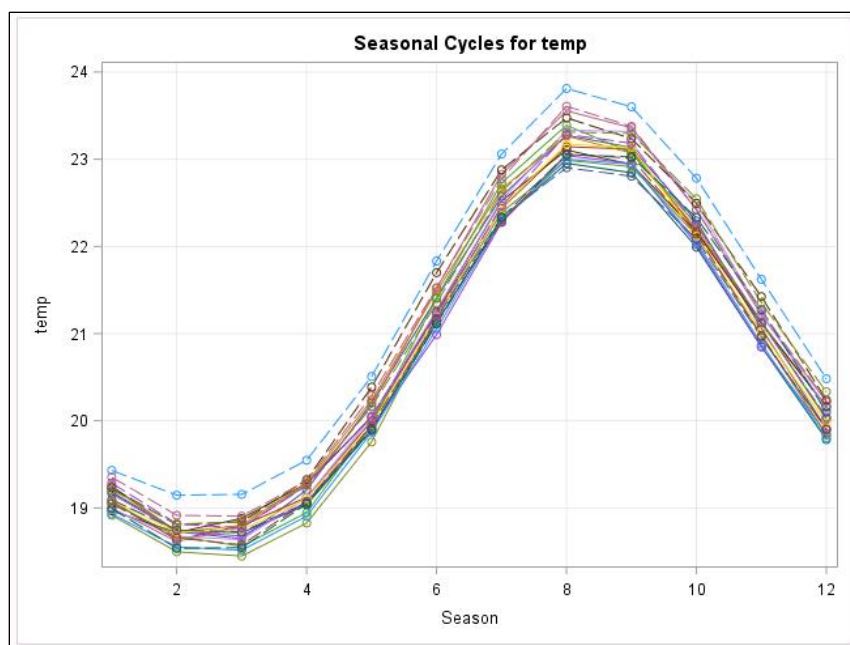
Selected Output

The output begins with a few tables that display information about the data set as well as the time series variable.

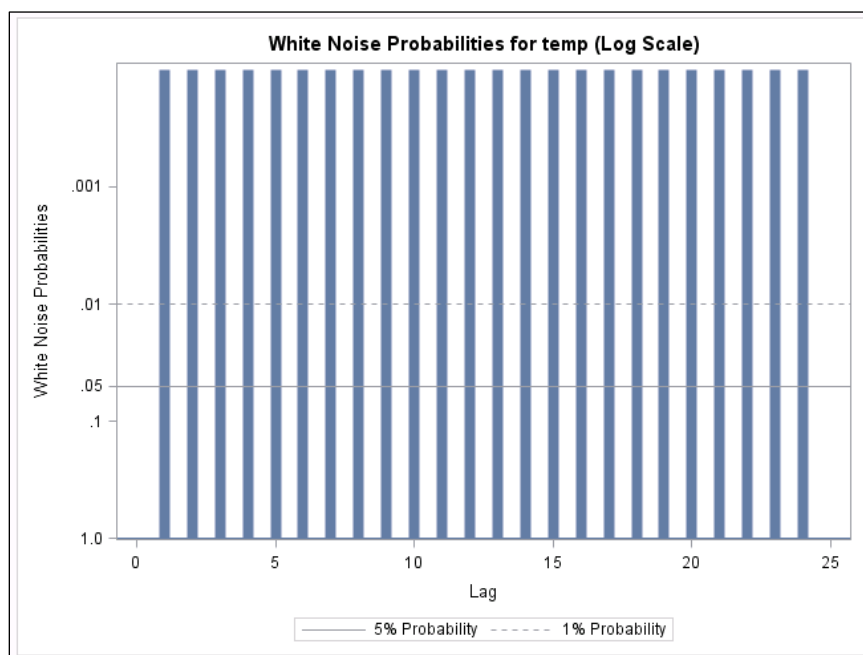


The time series plot provides a visual perspective of the dependent variable **temp** across the time ID variable **Datechk**. From this plot, you can see a clear seasonality to the data. However, it is difficult to determine whether there is a trend.

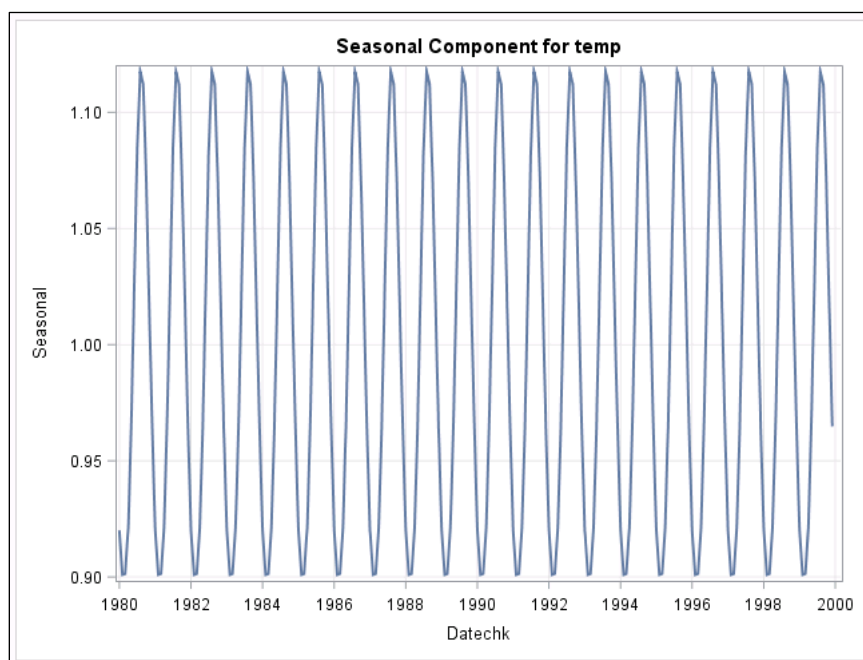
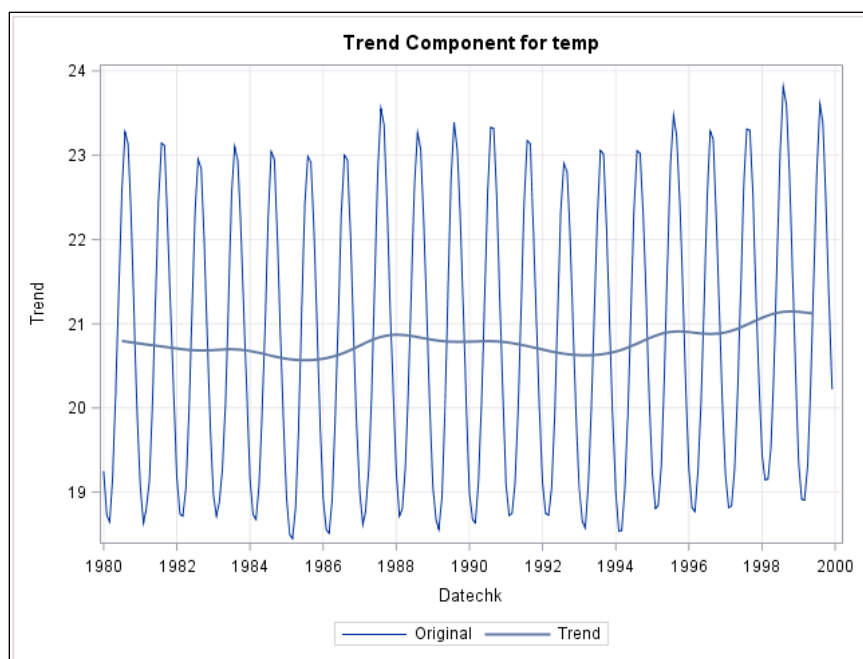
The series plot also gives an impression of whether the seasonality is additive or multiplicative. In the plot, notice that the peaks and valleys of the seasonal fluctuation appear consistent. This indicates an additive seasonality. If this plot showed peaks and valleys of the fluctuations becoming closer together or farther apart as time passed, this would indicate a multiplicative seasonality.



The seasonal cycles image displays a line for each year of the data. From this output, you can determine that each year followed a similar seasonal cycle. If there were lines that did not follow the pattern of the others, this could represent years when potentially some departure from the typical seasonal pattern existed. In this case, you see that the SST reached its lowest near March and its highest near August.



When the white noise probabilities are very small (p -values < 0.0001), it might be more advantageous to look at them using the log scale image. In either case, you can deduce that the series is not composed of only white noise. There is some attribute that can be modeled. What are these attributes? What components can you use in the modeling for this series?



Output from the decomposition analysis assists in the determination of the presence of a trend component, seasonal component, or both. From the trend component plot, you see that the trend line is rather flat. This suggests the lack of a trend in this series. From the seasonal component plot, you see a definitive seasonality that ranges from 10% below the mean at its lowest to slightly more than 10% above the mean at its highest. From these plots, you can narrow the focus to an exponential smoothing model that would incorporate seasonality but not trend.

In this case, from what you saw in the series plot earlier, this is either an additive seasonal ESM or the additive Winters model.

Note: With the exploration complete, you can proceed to the Modeling and Forecasting task.

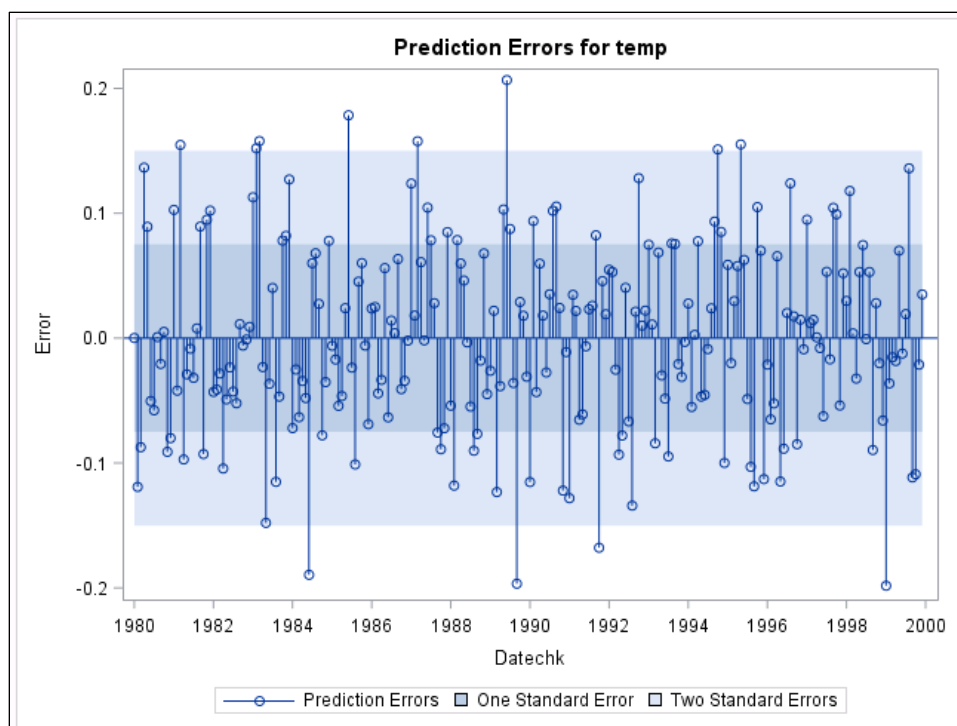
11. Exit the Maximized View. This causes the left-side navigation panel to reappear.
12. Double-click the **Modeling and Forecasting** task to open it. Click the **Maximized View** button.
13. On the DATA tab, select the **SST** data set as the data set of interest. Click the plus sign (+) next to **Dependent variables**. Select **temp**. Click **OK**. Click and expand **Additional Roles**.
14. Click the plus sign (+) next to **Time ID**. Select **Datechk**. Click **OK**. The Interval area in the properties is automatically updated.
15. Click the **MODEL** tab. Next to Forecasting model type, select **Exponential smoothing** in the drop-down box. Under Model Settings, select **Additive seasonal exponential smoothing** for the Forecasting model. Do not make changes to the transformation or to the forecast type.
16. Click and expand **Plots**. Do not change the **Select plots to display** field. Keep **Default plots**.
17. Click the **OPTIONS** tab. Select **12** as the number of periods to forecast. (That is one year.)
18. Click the **OUTPUT** tab. Select the **Create fit statistics data set** check box. Do *not* change the name of the **outstat** data set.

Note: Alternatively, you can write the SAS code directly as follows:

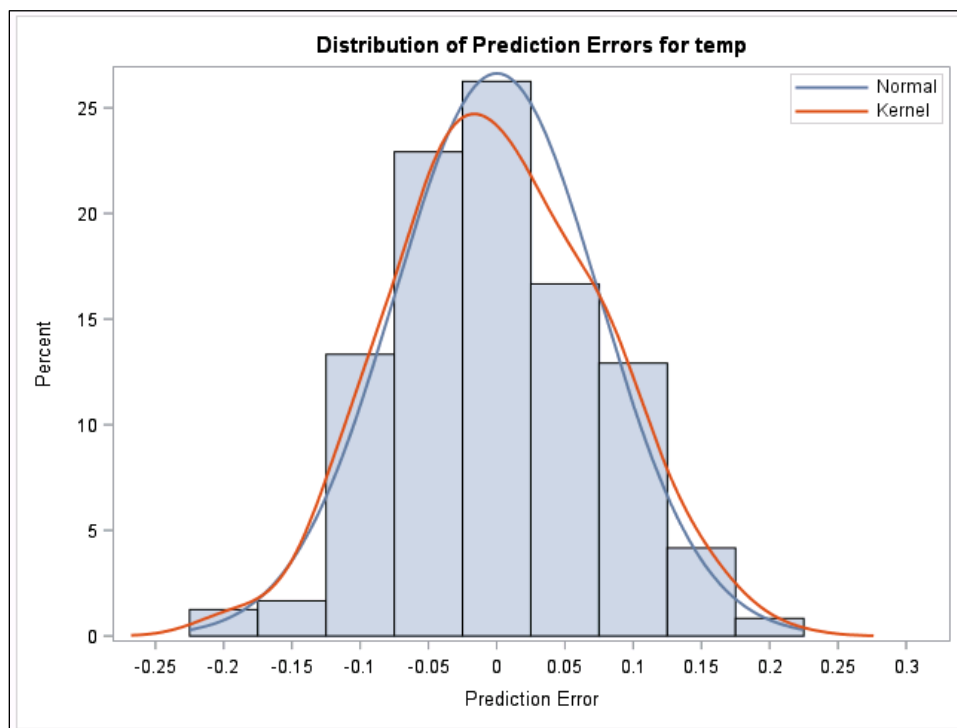
```
*Additive Seasonal Model;
proc esm data=STSM.SST
    back=0 lead=12
    seasonality=12
    plot=(corr errors modelforecasts)
    outstat=WORK.outstat;
    id Datechk interval=month;
    forecast temp / alpha=0.05 model=addseasonal;
run;
```

19. Click the running person icon to run this task.

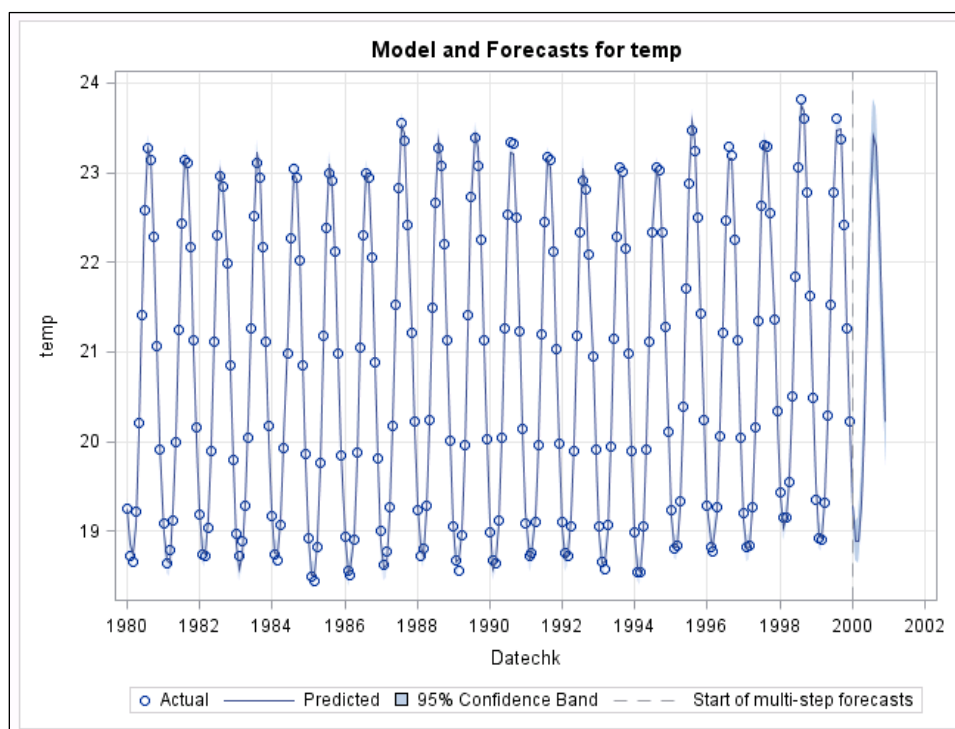
Selected Output



From the prediction errors image, you can see that the prediction error for the **temp** series generally falls within one or two standard errors from zero. A few points escape this region, but not enough to cause concern.



The distribution of prediction errors for the **temp** series is shown in a histogram with a superimposed normal and kernel curve. You can see that the errors seem to be approximately normal.



You are also provided with a model and forecasts series plot for **temp**. You can see that the additive seasonal ESM does a good job of picking up the seasonality of the time series and carries this into the future forecasts.

To check the fit statistics of this additive seasonal model, examine the **outstat** data set that you asked SAS to generate. Do this by clicking the **OUTPUT DATA** tab next to the RESULTS tab.

CODE	LOG	RESULTS	OUTPUT DATA
Table:	WORK.OUTSTAT	View:	Column names
Columns	Total rows: 1 Total columns: 57		
☒ Select all	Rows 1-1		
☒ _NAME_			
☒ REGION			
	AIC	AICC	SBC
	-1240.933593	-1240.88296	-1233.972315

Scrolling over, you see many statistics that were generated from the model. You can look at any of them, but focus on the AIC (-1240.934) and SBC (-1233.972). There is another additive model that could be chosen for the time series. This is the additive Winters model. Run an additive Winters model and compare the AIC and SBC to this model.

Note: Alternatively, you can run PROC PRINT.

```
proc print data=WORK.outstat;
    var AIC SBC;
run;
```

Obs	AIC	SBC
1	-1240.93	-1233.97

20. Click the **Model** tab and look under **Model Settings**. Change the **Forecasting model** drop-down selection to the **Winters additive** method.
21. Under the OUTPUT tab, change the name of the fit statistics data set to **outstat2**. This enables you to retain both output data sets.

Note: Alternatively, you can write the SAS code directly as follows:

```
*Additive Winters model;
proc esm data=STSM.SST
    back=0 lead=12
    seasonality=12
    plot=(corr errors modelforecasts)
    outstat=WORK.outstat2;
id Datechk interval=month;
forecast temp / alpha=0.05 model=addwinters;
run;
```

22. Rerun the model by clicking the running person icon.
23. Like the additive seasonal model, you can look at the provided images. However, click the **OUTPUT DATA** tab and look at the **outstat2** data set.

CODE

LOG

RESULTS

OUTPUT DATA

Table: WORK.OUTSTAT2
View: Column names
Filter: (none)

Columns
Total rows: 1 Total columns: 57

	AIC	AICC	SBC
☒ Select all ☒ 123 ADJRSQ	-1238.097194	-1237.995499	-1227.655277

If you compare the AIC and SBC from this model, -1238.097 and -1227.655 respectively, to the additive seasonal model, the additive seasonal model has slightly smaller values. Smaller is better.

Note: Alternatively, you can run PROC PRINT.

```
proc print data=WORK.outstat2;
    var AIC SBC;
run;
```

Obs	AIC	SBC
1	-1238.10	-1227.66

Note: In both the additive seasonal model and the additive Winters model, the white noise image shows that there might still be something beyond white noise that could be incorporated into the model. This might suggest the inclusion of a predictor variable.
For this, you leave the ESM methodology and incorporate another method.

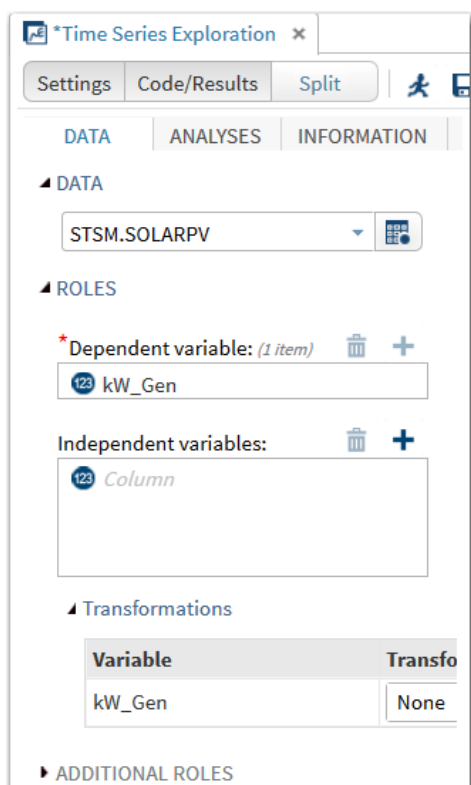
End of Demonstration

Determining Autoregressive Order (ARIMA Models)

The series **stsm.solarpv** is analyzed to determine whether there is an autoregressive process in **kW_Gen** and, if so, in what order.

Create a new Time Series Exploration task in SAS Studio.

1. On the DATA tab, select the data set **SolarPV**. Then, select **kW_Gen** as the dependent variable.



- Click the triangle next to ADDITIONAL ROLES. then select **EDT** as the time ID and accept the properties that are populated. SAS recognizes **EDT** as weekly.

ADDITIONAL ROLES

Time ID: (1 item)

EDT

Properties

Interval: Week

Multiplier: 1

Shift: 1

Season length: 52

Group analysis by:

Column

The generated SAS syntax is shown below.

```
proc sort data=STSM.SOLARPV out=Work.preProcessedData;
    by EDT;
run;

proc timeseries data=Work.preProcessedData seasonality=52
    plots=(series corr);
    id EDT interval=week;
    var kW_Gen / accumulate=none transform=none dif=0 sdif=0;
run;
```

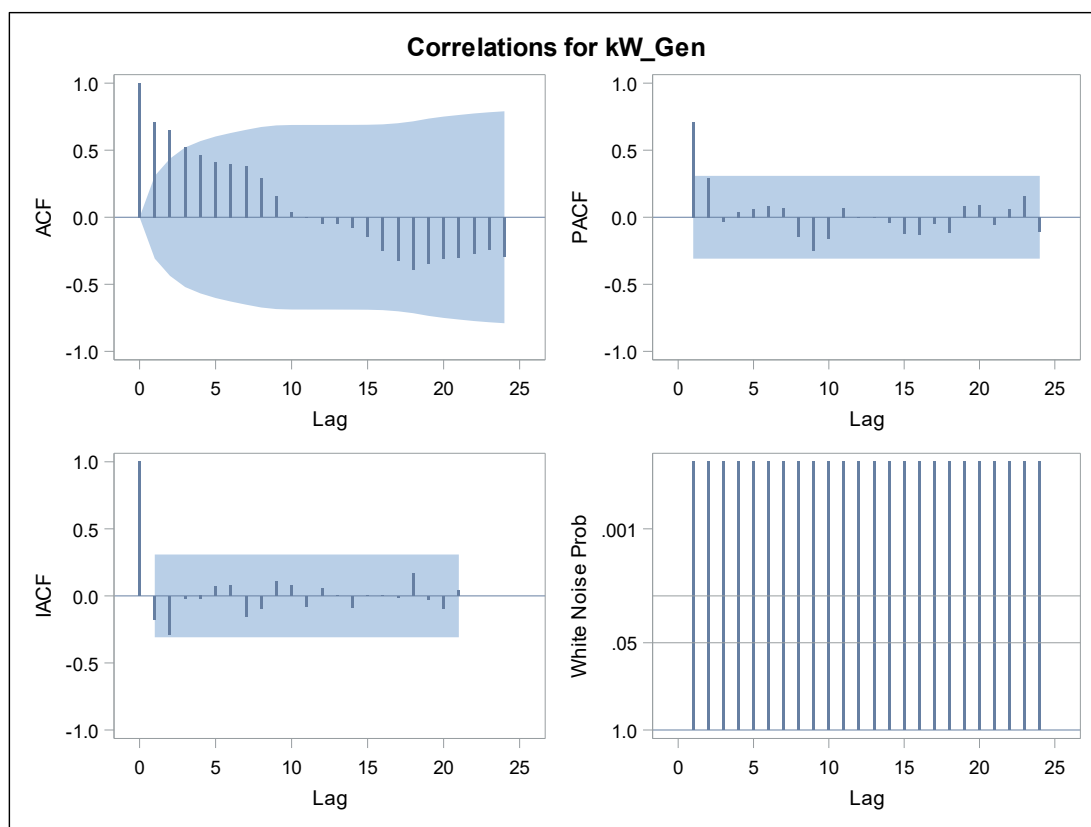
Note: Alternatively, you can write the code directly in SAS/ETS.

```
proc timeseries data=STSM.SOLARPV plots=(corr);
    id EDT interval=week;
    var kW_Gen;
run;
```

- Submit the code.

Note: Some of the output is the same as in a previous demonstration and is not displayed here.

The ACF plot shows a pattern of gradually declining autocorrelation as lags increase. The PACF plot gives a clearer picture of the true autoregressive order, because it removes the proximity effect. The first lag is the only clearly significant lag, which implies an autoregressive order of 1.



Note: The IACF is the inverse autocorrelation function. If the model is a pure autoregressive model, then the IACF is an ACF that corresponds to a pure moving average model. It cuts off sharply when the lag is greater than p . This behavior is similar to the behavior of the partial autocorrelation function (PACF).

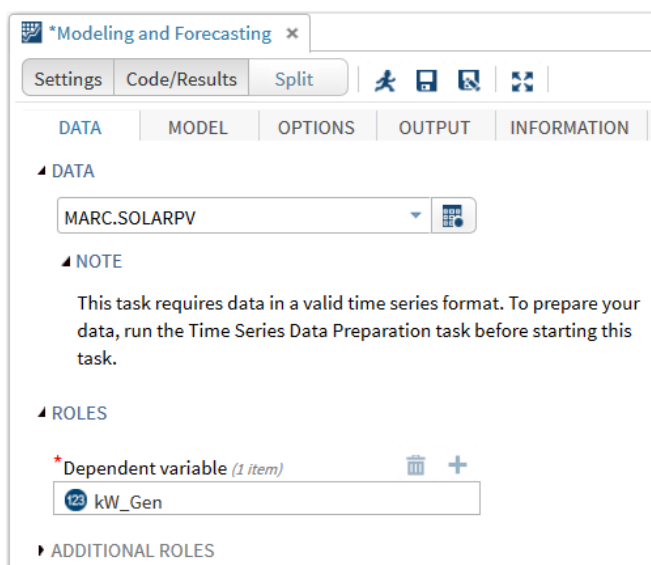
End of Demonstration

Estimation of an ARIMAX Model

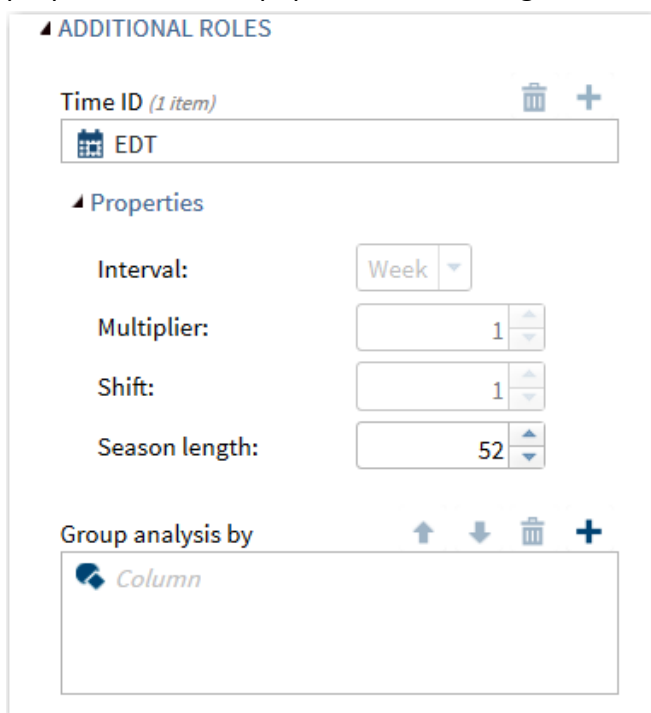
Estimate an AR(1) model for the **SolarPV** data set. Check the residual series to see whether it is white noise. Display the goodness-of-fit statistics for comparison with future models.

Use the kilowatts generated (**kW_Gen**) time series.

1. Create a new Modeling and Forecasting task in SAS Studio.
2. On the DATA tab, select **SolarPV** as the data set. Select **kW_Gen** as the dependent variable.



3. Click the triangle next to ADDITIONAL ROLES. Then select **EDT** as the time ID and accept the properties that are populated. SAS recognizes **EDT** as weekly.



4. On the **MODEL** tab, select **ARIMA** as the forecasting model type. Model settings appear. Select **1** in the **Autoregressive order (p)** field under ARIMA.

DATA MODEL OPTIONS OUTPUT INFORMATION

▲ MODEL

*Forecasting model type: ARIMA

▲ Model Settings

▲ ARIMA

Autoregressive order (p): 1

Differencing order (d): 0

Moving average order (q): 0

▲ Seasonal ARIMA

Autoregressive order (P): 0

Differencing order (D): 0

Moving average order (Q): 0

☒ Include intercept in model

► Plots

5. Expand **Plots** and click **Selected plots**.

▲ Plots

Select plots to display: Default plots

Default plots

Selected plots

All plots

No plots

- Clear the **Panels of cross-correlations plots** check box under Series Plots and the **One-step-ahead and multistep-ahead forecasts** check box under Forecast Plots.

The screenshot shows the 'Plots' section of the SAS Forecasting software. The 'Panels of cross-correlations plots' checkbox is unchecked. The 'One-step-ahead and multistep-ahead forecasts' checkbox is also unchecked.

- On the OPTIONS tab, set the **Number of periods to forecast** field to **0** under FORECAST SETTINGS. Clear the **Perform outlier detection** check box under OUTLIER DETECTION.

The screenshot shows the 'OPTIONS' tab of the SAS Forecasting software. The 'Number of periods to forecast' field is set to 0. The 'Perform outlier detection' checkbox is unchecked.

The generated SAS syntax is shown below.

```
/* ARIMA or ARIMAX */
proc sort data=STSM.SOLARPV out=Work.preProcessedData;
    by EDT;
run;

proc arima data=Work.preProcessedData plots
    (only)=(series(corr) residual(corr normal) );
    identify var=kW_Gen;
    estimate p=(1) method=ML;
    forecast lead=0 back=0 alpha=0.05 id=EDT interval=week;
run;
quit;
```

Note: Alternatively, you can write the SAS/ETS code directly.

```
proc arima data=STSM.SOLARPV
    plots(only)=(series(corr)
                    residual(corr normal));
    identify var=kW_Gen;
    estimate p=(1) method=ML;
quit;
```

8. Submit the code.

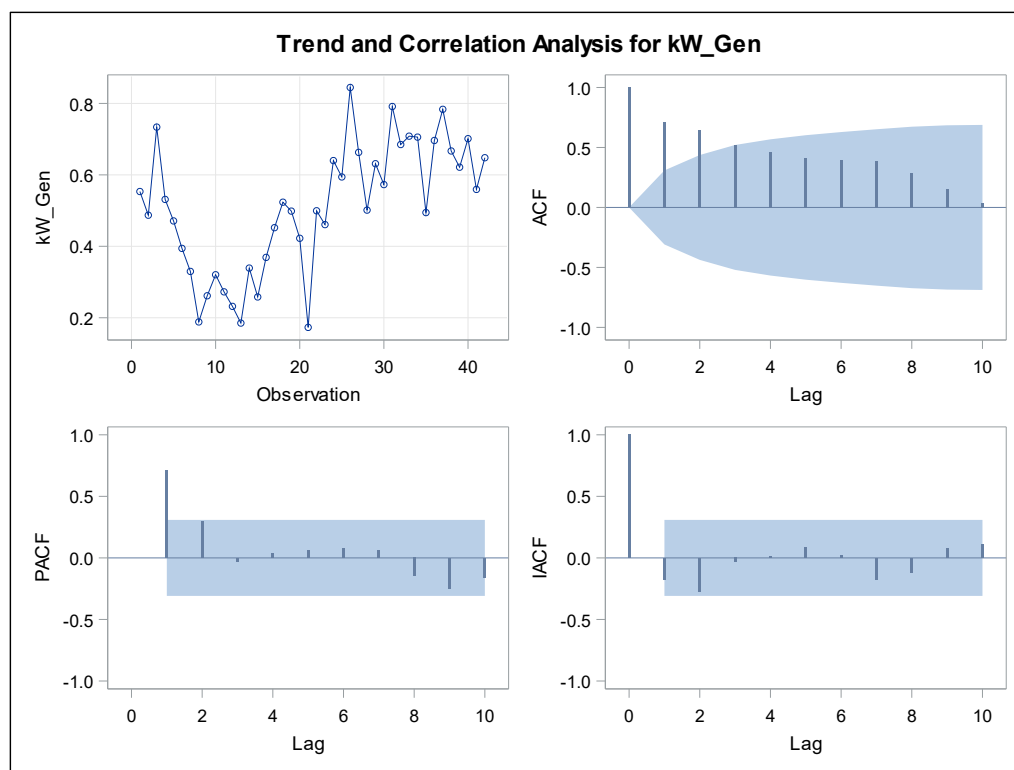
The output starts with basic statistics for the time series.

Name of Variable = kW_Gen	
Mean of Working Series	0.511078
Standard Deviation	0.179364
Number of Observations	42

The autocorrelations test for the first six lags shows statistical significance. This means a rejection of the white noise null hypothesis. Notice that the autocorrelation at the first lag is 0.709. That value slowly declines over sequential lags.

Autocorrelation Check for White Noise									
To Lag	Chi-Square	DF	Pr > ChiSq	Autocorrelations					
6	81.65	6	<.0001	0.709	0.648	0.519	0.460	0.412	0.396

These plots were seen in the previous section. The order 1 autoregressive model seems appropriate for these data.



Here the maximum likelihood parameters for the AR(1) model are displayed. The Parameter AR1,1 at Lag 1 is estimated as 0.70389. That is statistically significant, with a p -value less than .0001.

Maximum Likelihood Estimation					
Parameter	Estimate	Standard Error	t Value	Approx Pr > t	Lag
MU	0.52019	0.06309	8.25	<.0001	0
AR1,1	0.70389	0.11038	6.38	<.0001	1

The AIC and SBC values are displayed, but recall that these values are useful only when you compare two models of the same data.

Constant Estimate	0.154036
Variance Estimate	0.016529
Std Error Estimate	0.128564
AIC	-50.4857
SBC	-47.0103
Number of Residuals	42

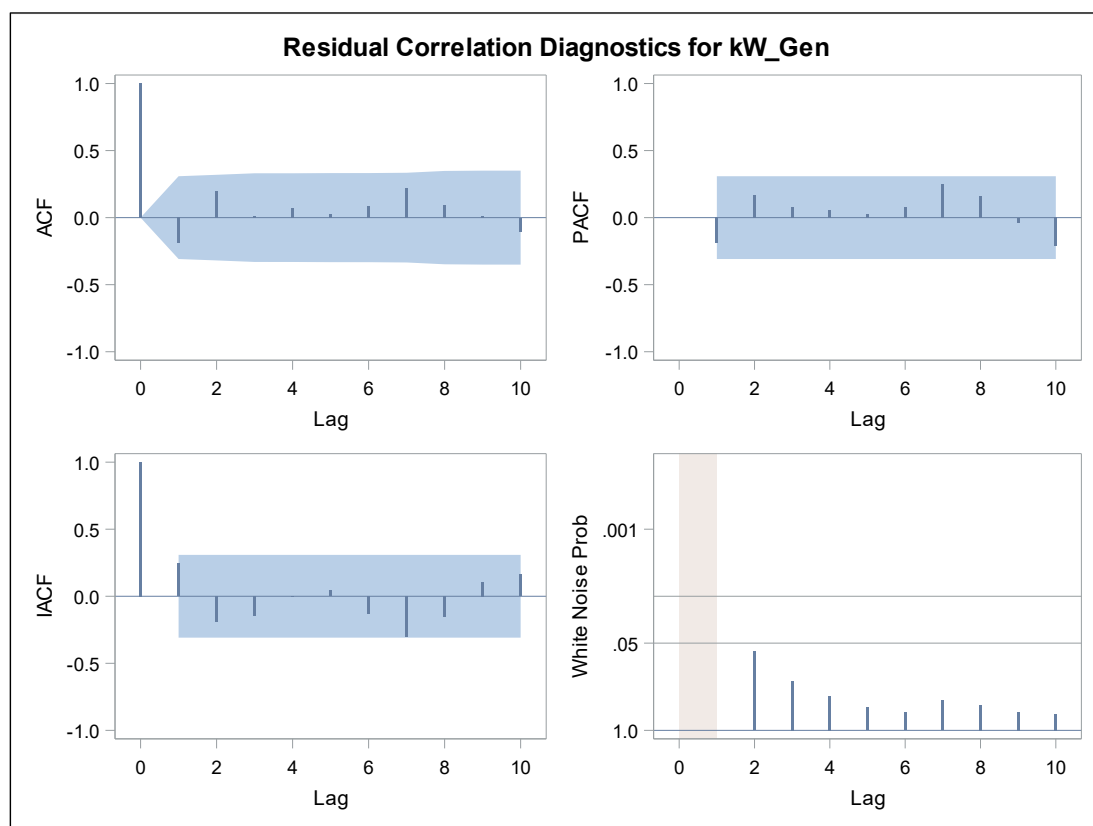
The correlations of parameter estimates can be used to check the multicollinearity of parameters. In this case, the correlation, as expected, is very low at 0.055.

Correlations of Parameter Estimates		
Parameter	MU	AR1,1
MU	1.000	0.055
AR1,1	0.055	1.000

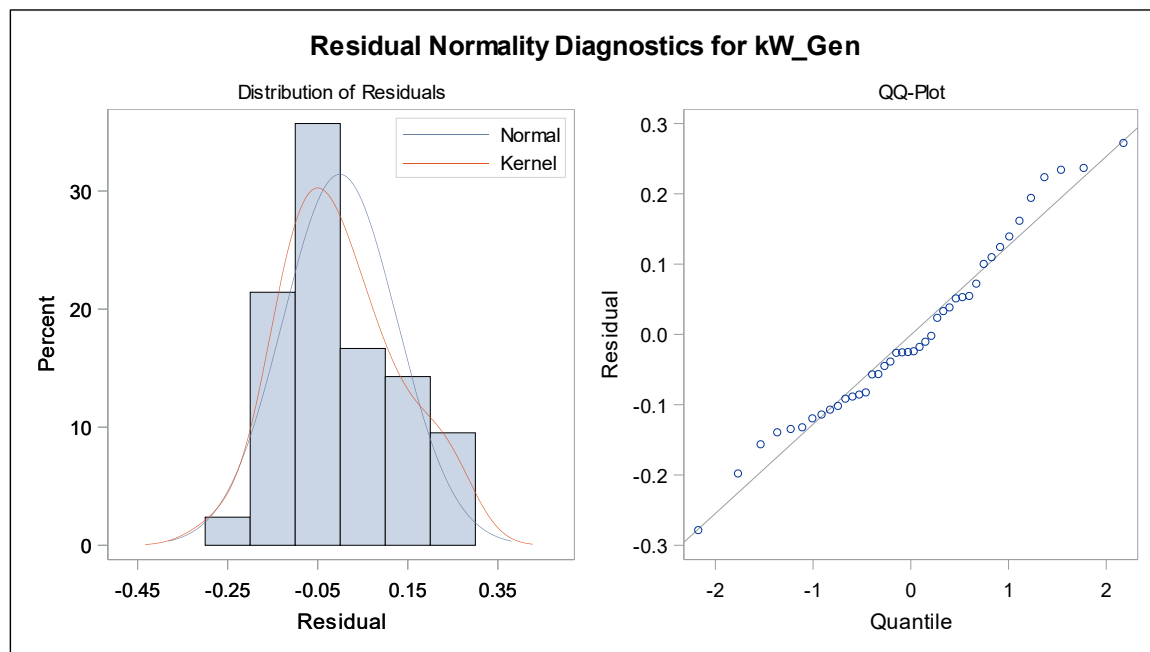
If this model is correct, the residuals from it should be white noise. The autocorrelation check of the first 24 lags indicates no particularly strong autocorrelation, although one value is above 0.2 in absolute value.

Autocorrelation Check of Residuals									
To Lag	Chi-Square	DF	Pr > ChiSq	Autocorrelations					
6	4.04	5	0.5430	-0.187	0.197	0.011	0.070	0.024	0.089
12	7.81	11	0.7301	0.221	0.090	0.009	-0.103	0.023	-0.039
18	13.25	17	0.7193	0.024	0.071	-0.000	-0.103	-0.027	-0.235
24	15.41	23	0.8792	-0.020	-0.010	-0.066	-0.077	0.043	-0.099

The residual auto-correlation plots, and the white noise plot of the residuals, confirm that the residual autocorrelations are not statistically significant.



Another white noise assumption is that the residuals are normally distributed. The histogram and QQ plot show the residuals to be reasonably normal (Gaussian normal).



The estimated mean of the model is reiterated. The estimate for the autoregressive parameter is expressed in a particular model formulation.

Model for variable kW_Gen	
Estimated Mean	0.520193

Autoregressive Factors	
Factor 1:	1 - 0.70389 B**(1)

End of Demonstration

Using Input Variables in ARIMAX Models

Look at the relationship between **kW_Gen** and **Cloud_Cover**.

1. Use the Time Series Exploration task.
2. On the DATA tab, select the data set **SolarPV**. Then, select **kW_Gen** as the dependent variable and **Cloud_Cover** and **cosval** (a series generated as a cosine wave with a yearly cycle) as the independent variables. Expand **ADDITIONAL ROLES** so that you can select **EDT** as the time ID variable.

The screenshot shows the 'DATA' tab of a software interface. The 'DATA' section has a dropdown menu set to 'STSM.SOLARPV'. Below it, the 'Filter' is set to '(none)'. The 'ROLES' section shows 'kW_Gen' as the 'Dependent variable' and 'Cloud_Cover' and 'cosval' as 'Independent variables'. The 'Transformations' section contains a table with columns 'Variable', 'Accumula...', 'Transformation', and 'Simple Differ'. The table has three rows: 'kW_Gen', 'Cloud_Cover', and 'cosval', all with 'None' in the 'Accumula...' and 'Transformation' columns, and '0' in the 'Simple Differ' column. The 'ADDITIONAL ROLES' section shows 'EDT' as the 'Time ID'. The 'Properties' section shows 'Interval' set to 'Week', 'Multiplier' set to '1', and 'Shift' set to '1'.

Variable	Accumula...	Transformation	Simple Differ
kW_Gen	None	None	0
Cloud_Cover	None	None	0
cosval	None	None	0

3. On the ANALYSES tab, clear the **Time Series** check box under SERIES PLOTS. Also, clear the **Perform autocorrelation analysis** check box.

The screenshot shows the 'ANALYSES' tab of the software interface. Under 'SERIES PLOTS', the 'Time Series' checkbox is unchecked. Under 'STATISTICS', the 'Perform autocorrelation analysis' checkbox is unchecked. Other options like 'Series histogram', 'Seasonal cycles', 'CROSS-CORRELATION ANALYSIS', 'DECOMPOSITION ANALYSIS', 'SPECTRAL DENSITY ANALYSIS', and 'UNIT ROOT TEST ANALYSIS' are listed but not expanded.

- Expand the **CROSS-CORRELATION ANALYSIS** section. Select the **Perform cross-correlation analysis** check box. Plot suboptions appear. The **Cross-series** check box is already selected. Select the **Cross-correlation function** check box.

The screenshot shows the SAS software interface with the **ANALYSES** tab selected. The **CROSS-CORRELATION ANALYSIS** section is expanded, revealing several options. The **Perform cross-correlation analysis** checkbox is checked. Under the **Plots** sub-section, the **Cross-series** and **Cross-correlation function** checkboxes are checked, while the **Normalized cross-correlation function** checkbox is unchecked. The **Number of lags** dropdown menu is set to **Use default value**.

The generated SAS syntax is shown below.

```
proc sort data=STSM.SOLARPV out=Work.preProcessedData;
    by EDT;
run;

proc timeseries data=Work.preProcessedData seasonality=52
    crossplots=(series ccf);
    id EDT interval=week;
    var kW_Gen / accumulate=none transform=none dif=0 sdif=0;
    crossvar Cloud_Cover / accumulate=none transform=none dif=0
                sdif=0;
    crossvar cosval / accumulate=none transform=none dif=0 sdif=0;
    ods exclude CCFNORMPlot;
run;
```

Note: Alternatively, you can write the SAS/ETS code directly.

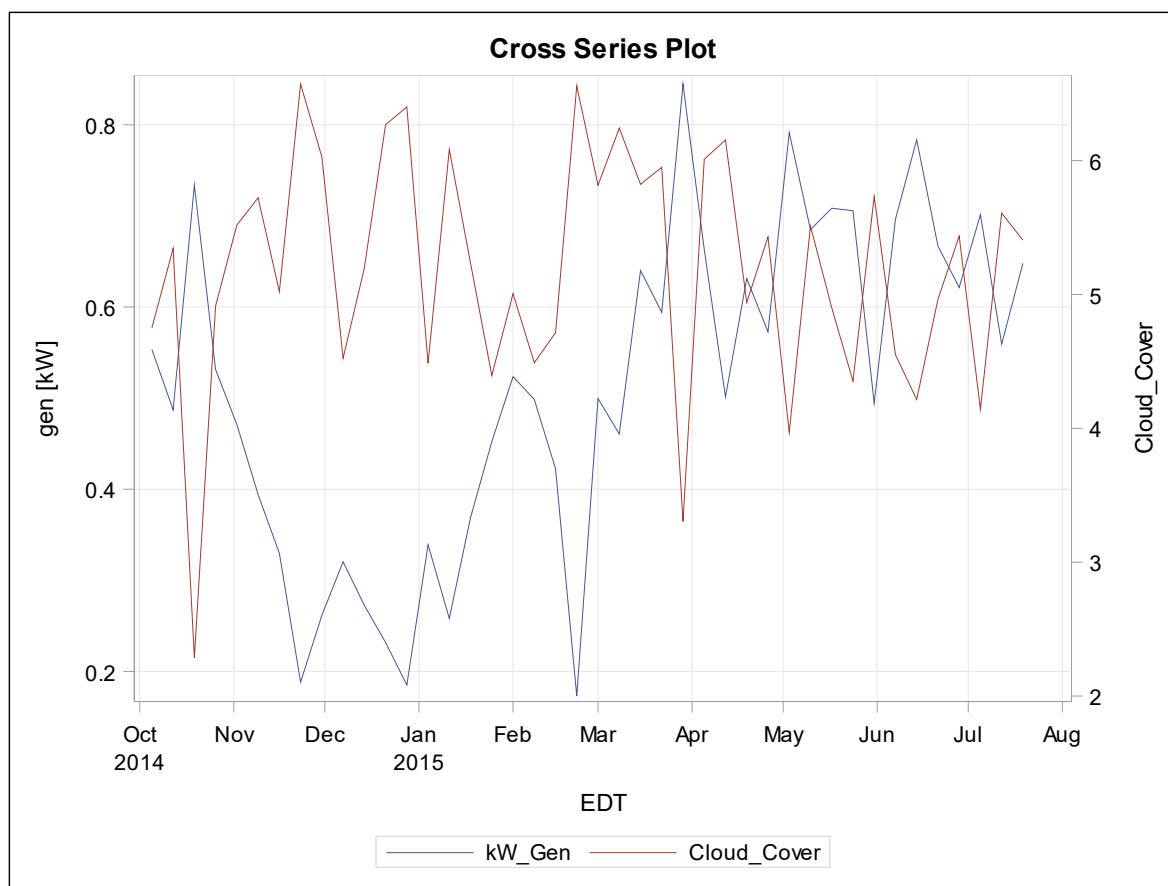
```
proc timeseries data=STSM.SOLARPV
                crossplots=(series ccf);
  id EDT interval=week;
  var kW_Gen;
  crossvar Cloud_Cover cosval;
  ods exclude CCFNORMPlot;
run;
```

5. Submit the code.

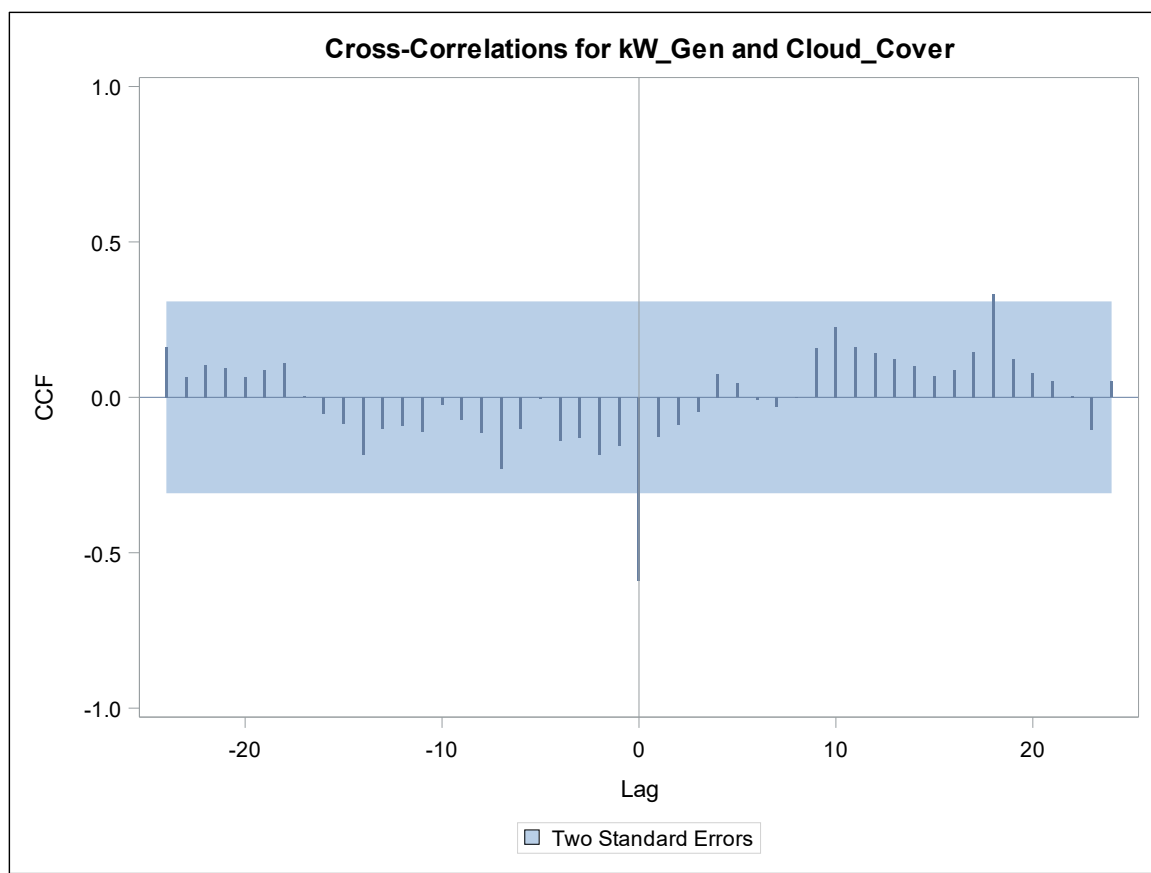
Input Data Set	
Name	WORK.PREPROCESSEDDATA
Label	
Time ID Variable	EDT
Time Interval	WEEK
Length of Seasonal Cycle	52

Variable Information	
Name	kW_Gen
Label	
First	Sun, 5 Oct 2014
Last	Sun, 19 Jul 2015
Number of Observations Read	42

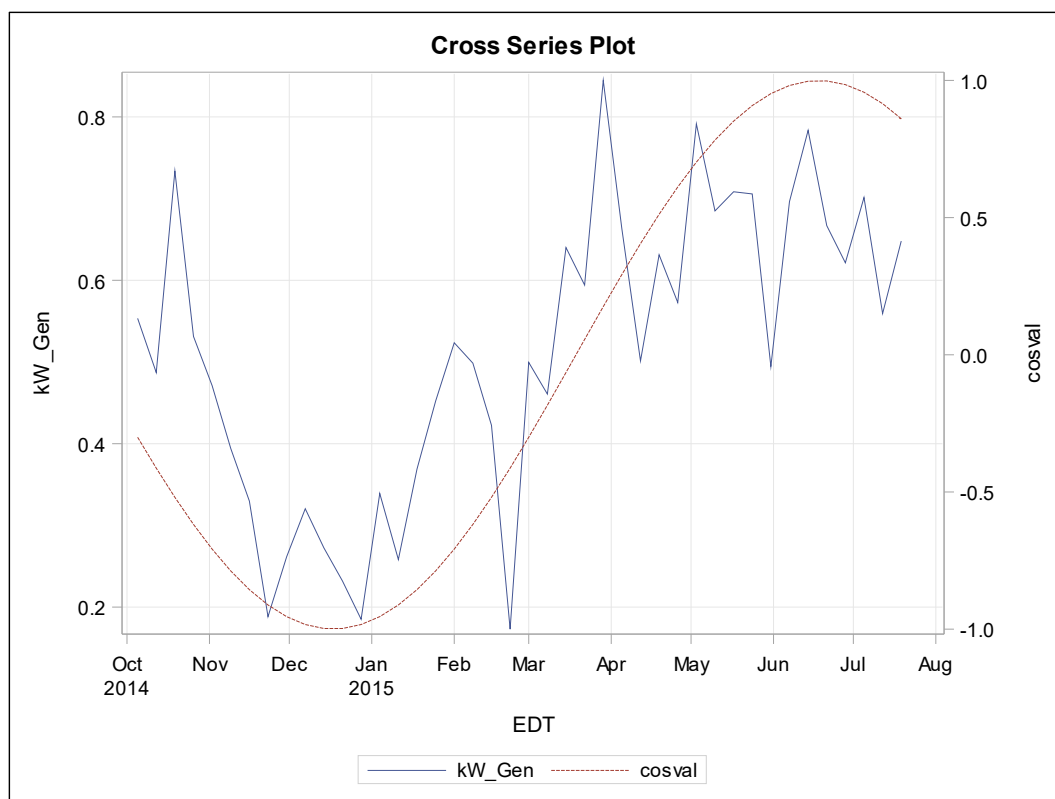
The cross series plot with **Cloud_Cover** seems to show an inverse relationship between cloud cover and generated solar power. Where cloud cover increases, there appears to be a concurrent decrease in the generated power.



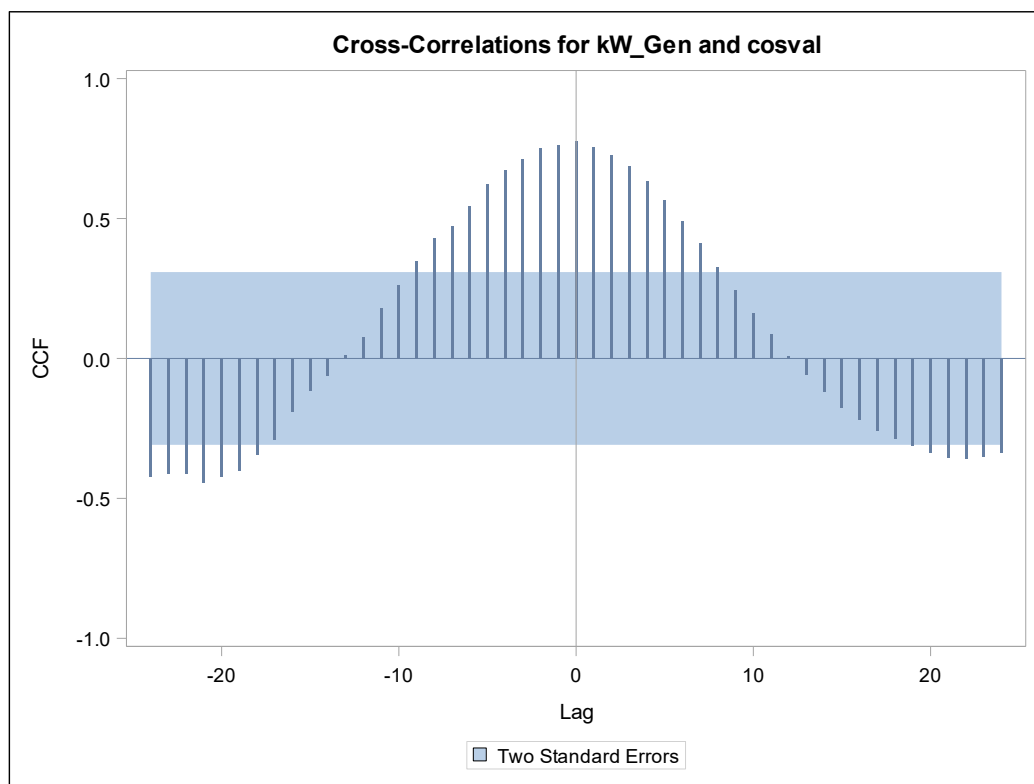
The cross-correlations plot shows modest correlations at many positive lags and a large and statistically significant one at the 0 lag, which is the concurrent time. This indicates that **Cloud_Cover** might make an important contribution to modeling and forecasting the kilowatts that are generated.



There is no way to model seasonality in the kW_Gen series because there is less than a seasonal cycle's worth of data. A strategy might be to model solar power generation as a function of a cosine wave, starting at the summer solstice. This idea comes from the knowledge that the sun's rays are strongest in the northern hemisphere on the summer solstice and weakest on the winter solstice. This seems to be borne out in the sample as the cross-series plot shows that the series are following each other nicely.



The shape of the cross-correlation function plot reflects the nature of the cosine function.



End of Demonstration

Estimation of Input Variables in ARIMAX Models

Estimate an ARMAX(1,0) model for the **SolarPV** data set with **Cloud_Cover** as an exogenous effect. Check the residual series to see whether it is white noise. Display the goodness-of-fit statistics for comparison with the AR(1) model that was previously estimated.

Use the kilowatts generated (kW_Gen) time series.

1. Create a new Modeling and Forecasting task in SAS Studio.
2. On the DATA tab, select **SolarPV** as the data set. Select **kW_Gen** as the dependent variable.
3. Click the triangle next to ADDITIONAL ROLES. Then select **EDT** as the time ID and accept the properties that are populated. SAS recognizes **EDT** as weekly.
4. On the MODEL tab, select **ARIMAX** as the forecasting model type. Model settings appear. Select **1** in the **Autoregressive order (p)** field under ARIMA.
5. Under Independent variables, add **Cloud_Cover**.
6. Expand **Plots** and click **Selected plots**.
7. Clear the **Panels of correlations plots** and **Panels of cross-correlation plots** check boxes under Series Plots. Also clear the **One-step-ahead and multistep-ahead forecasts** check box under Forecast Plots.
8. On the OPTIONS tab, set the **Number of periods to forecast** field to **0** under FORECAST SETTINGS. Clear the **Perform outlier detection** check box under OUTLIER DETECTION.

The generated SAS syntax is shown below.

```
proc sort data=STSM.SOLARPV out=Work.preProcessedData;
  by EDT;
run;

proc arima data=Work.preProcessedData plots
  (only)=(residual(corr normal) );
  identify var=kW_Gen crosscorr=(Cloud_Cover);
  estimate p=(1) input=(Cloud_Cover) method=ML;
  forecast lead=0 back=0 alpha=0.05 id=EDT interval=week;
run;
quit;
```

Note: Alternatively, you can write the SAS/ETS code directly.

Note: Use the  button to verify that you are in interactive mode.

```
proc arima data=STSM.SOLARPV
  plots(only)=(series(corr crosscorr)
    residual(corr normal));
  identify var=kW_Gen crosscorr=(Cloud_Cover);
  estimate p=(1) input=(Cloud_Cover) method=ML;
run;
```


9. Submit the code up to the first RUN statement.

The series identification portion of the output is not shown.

Both the autoregressive component and the component for **Cloud_Cover** are statistically significant ($p < .0001$).

The parameter estimate for **Cloud_Cover** indicates that for each unit increase in cloud cover for a week, the average daily production of solar power decreases by .0096 kilowatts. This value is statistically significant with a p -value less than .0001. Cloud cover seems to have a negative effect on solar production, and as the overlaid series plots from the previous demonstration imply.

Maximum Likelihood Estimation							
Parameter	Estimate	Standard Error	t Value	Approx Pr > t	Lag	Variable	Shift
MU	1.00001	0.08901	11.23	<.0001	0	kW_Gen	0
AR1,1	0.86587	0.07766	11.15	<.0001	1	kW_Gen	0
NUM1	-0.09061	0.0096050	-9.43	<.0001	0	Cloud_Cover	0

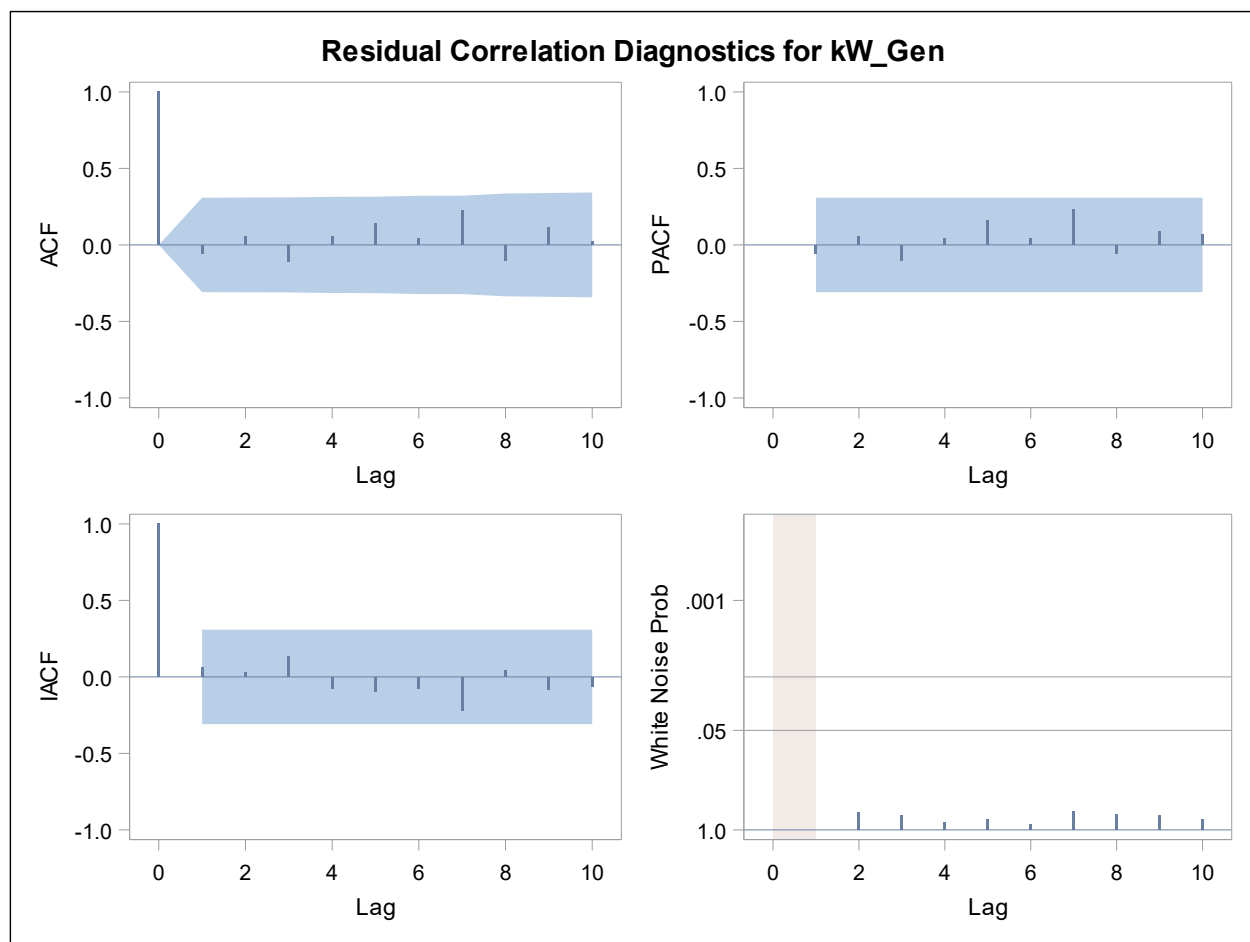
The AIC for this model is -95, which is lower (more negative) than the AR(1) model (AIC=-50). This model fits better than the AR(1) model.

Constant Estimate	0.134134
Variance Estimate	0.005503
Std Error Estimate	0.074179
AIC	-95.0433
SBC	-89.8303
Number of Residuals	42

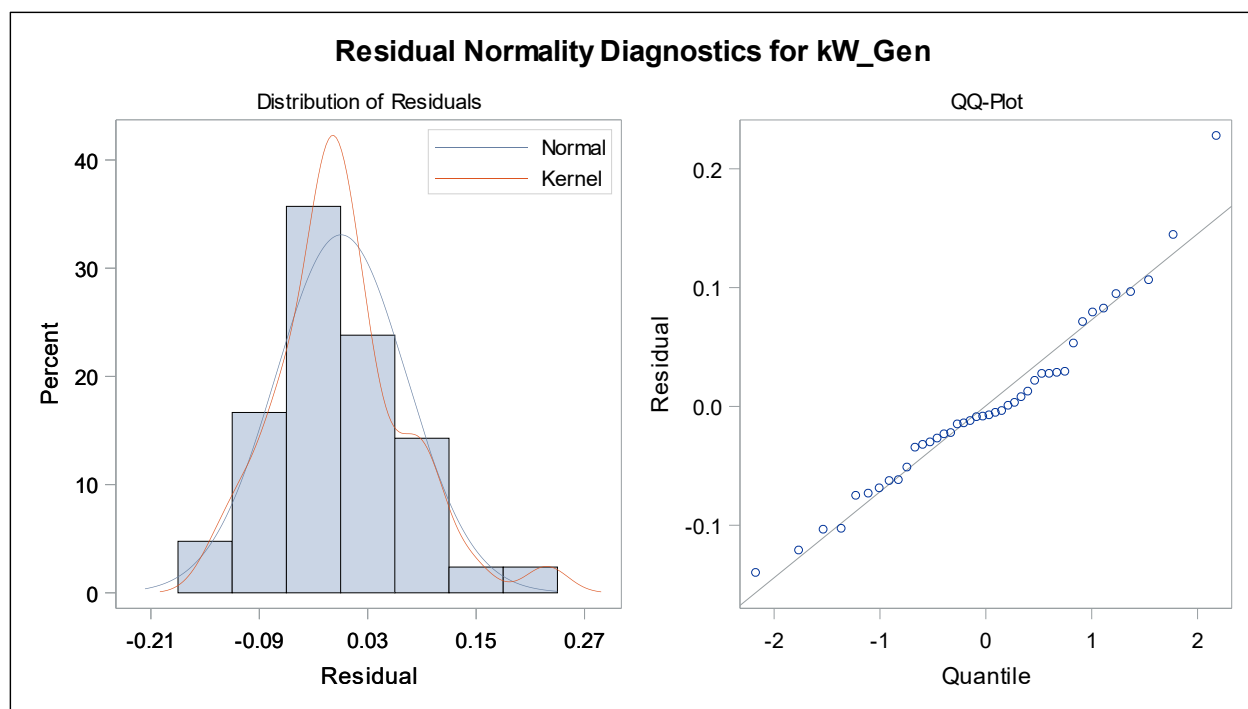
Correlations of Parameter Estimates			
Variable Parameter	kW_Gen MU	kW_Gen AR1,1	Cloud_Cover NUM1
kW_Gen MU	1.000	0.103	-0.553
kW_Gen AR1,1	0.103	1.000	0.033
Cloud_Cover NUM1	-0.553	0.033	1.000

The residuals seem to be a white noise series.

Autocorrelation Check of Residuals									
To Lag	Chi-Square	DF	Pr > ChiSq	Autocorrelations					
6	2.08	5	0.8379	-0.058	0.056	-0.109	0.057	0.139	0.043
12	7.16	11	0.7860	0.225	-0.101	0.112	0.024	-0.116	-0.071
18	14.98	17	0.5970	0.029	0.121	-0.006	-0.202	-0.212	0.086
24	16.33	23	0.8406	-0.033	0.030	-0.016	-0.051	-0.088	-0.040



The residuals are relatively normally distributed.



Model for variable kW_Gen	
Estimated Intercept	1.000009

Autoregressive Factors	
Factor 1:	1 - 0.86587 B**(1)

Input Number 1	
Input Variable	Cloud_Cover
Overall Regression Factor	-0.09061

10. If you are working with the SAS Studio task, reopen the task options, change the independent variable under the MODEL tab to **cosval**. (Hint: Put the **Cloud_Cover** variable in the trash and add **cosval**.) The generated code should look like the following:

```
proc arima data=Work.preProcessedData plots
  (only)=(residual(corr normal));
  identify var=kW_Gen crosscorr=(cosval);
  estimate p=(1) input=(cosval) method=ML;
  forecast lead=0 back=0 alpha=0.05 id=EDT interval=week;
run;
quit;
```

11. If you submit program code from Files and Folders, and you are still in interactive mode, you can highlight the following section of code from STSM03d03.sas:

```
estimate p=(1) input=(cosval) method=ML;
run;
```

12. Submit the new code.

The series identification portion of the output is not shown.

The regression component for **cosval** is statistically significant ($p < .0001$), but the AR(1) component is marginally non-significant ($p = 0.0566$). You could potentially test a model without that first-order autoregressive component to see if it fits better.

Correlation of kW_Gen and cosval	
Variance of input =	0.562007
Number of Observations	42

Maximum Likelihood Estimation							
Parameter	Estimate	Standard Error	t Value	Approx Pr > t	Lag	Variable	Shift
MU	0.52652	0.02430	21.67	<.0001	0	kW_Gen	0
AR1,1	0.29150	0.15292	1.91	0.0566	1	kW_Gen	0
NUM1	0.18493	0.03221	5.74	<.0001	0	cosval	0

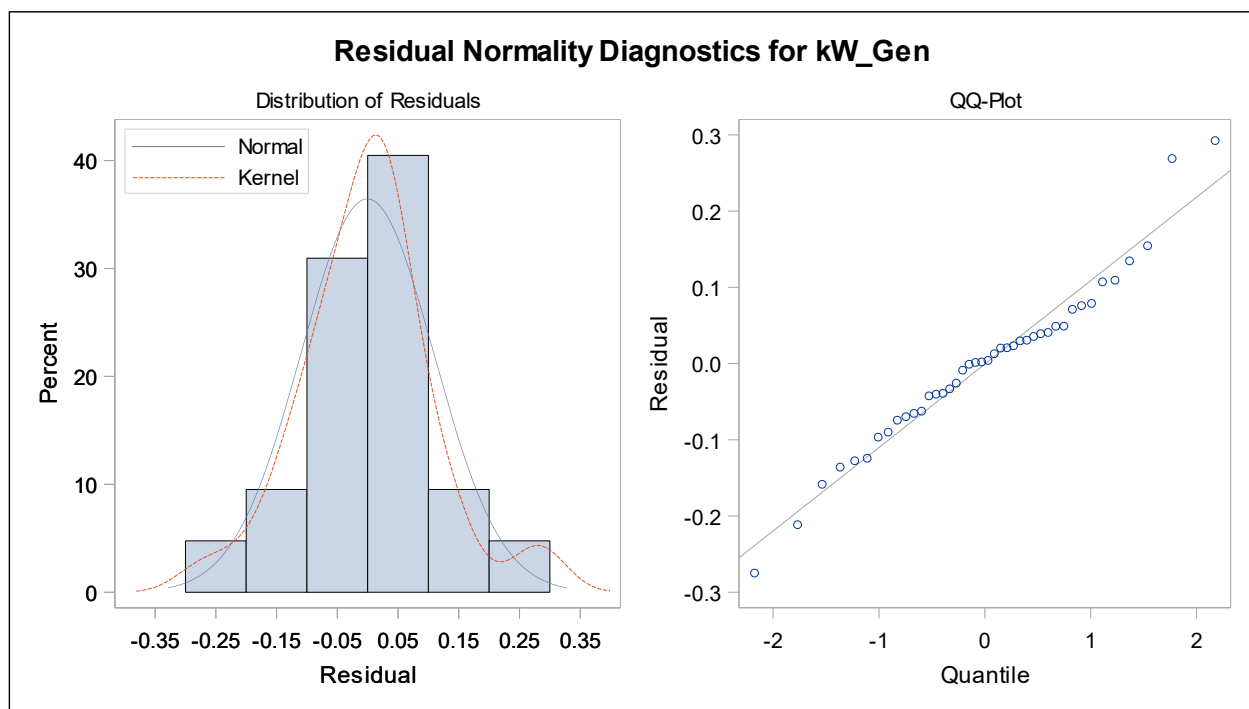
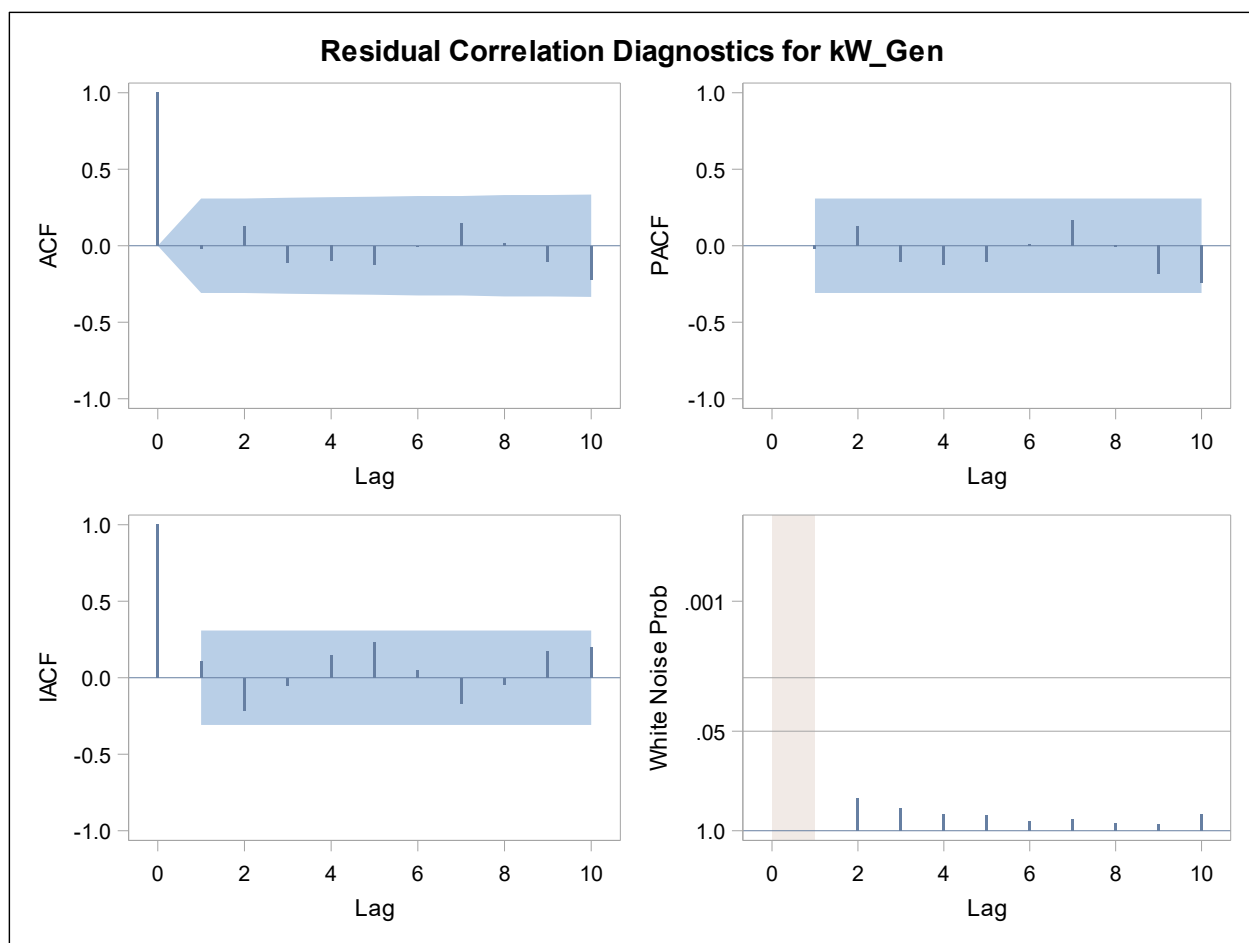
The AIC and SBC estimates for this model are slightly worse (less negative) than in the ARMAX(1) model with **Cloud_Cover**.

Constant Estimate	0.373035
Variance Estimate	0.012596
Std Error Estimate	0.112231
AIC	-61.5573
SBC	-56.3443
Number of Residuals	42

Correlations of Parameter Estimates				
Variable Parameter		kW_Gen MU	kW_Gen AR1,1	cosval NUM1
kW_Gen	MU	1.000	-0.012	0.098
kW_Gen	AR1,1	-0.012	1.000	-0.012
cosval	NUM1	0.098	-0.012	1.000

Autocorrelation Check of Residuals									
To Lag	Chi-Square	DF	Pr > ChiSq	Autocorrelations					
6	2.58	5	0.7640	-0.022	0.128	-0.110	-0.100	-0.122	-0.005
12	8.17	11	0.6984	0.149	0.017	-0.104	-0.221	-0.099	-0.077
18	14.26	17	0.6486	0.030	0.091	0.009	-0.118	-0.069	-0.232
24	17.54	23	0.7822	0.002	0.066	0.087	0.052	0.143	-0.010

The residuals for this model do not seem to be notably different from white noise.



Model for variable kW_Gen	
Estimated Intercept	0.526517

Autoregressive Factors	
Factor 1:	1 - 0.2915 B**(1)

Input Number 1	
Input Variable	cosval
Overall Regression Factor	0.184931

13. If you are working with the SAS Studio task, reopen the previous task option menus and add **Cloud_Cover** to **cosval** in the Independent variables list.

The generated code should now look like this:

```
proc arima data=Work.preProcessedData plots
  (only)=(residual(corr normal) );
  identify var=kW_Gen crosscorr=(cosval Cloud_Cover) ;
  estimate p=(1) input=(cosval Cloud_Cover) method=ML;
  forecast lead=0 back=0 alpha=0.05 id=EDT interval=week;
run;
quit;
```

Alternatively, if you are working with the STSM03d03.sas program, highlight the code:

```
estimate p=(1) input=(Cloud_Cover cosval) method=ML;
run;
```

14. Submit the code.

Correlation of kW_Gen and cosval	
Variance of input =	0.562007
Number of Observations	42

Correlation of kW_Gen and Cloud_Cover	
Variance of input =	0.775925
Number of Observations	42

The parameter estimates for the AR(1) component and both inputs are statistically significant ($p < 0.0001$).

Maximum Likelihood Estimation							
Parameter	Estimate	Standard Error	t Value	Approx Pr > t	Lag	Variable	Shift
MU	0.99744	0.05736	17.39	<.0001	0	kW_Gen	0
AR1,1	0.55574	0.13307	4.18	<.0001	1	kW_Gen	0
NUM1	0.16664	0.02985	5.58	<.0001	0	cosval	0
NUM2	-0.09091	0.01019	-8.92	<.0001	0	Cloud_Cover	0

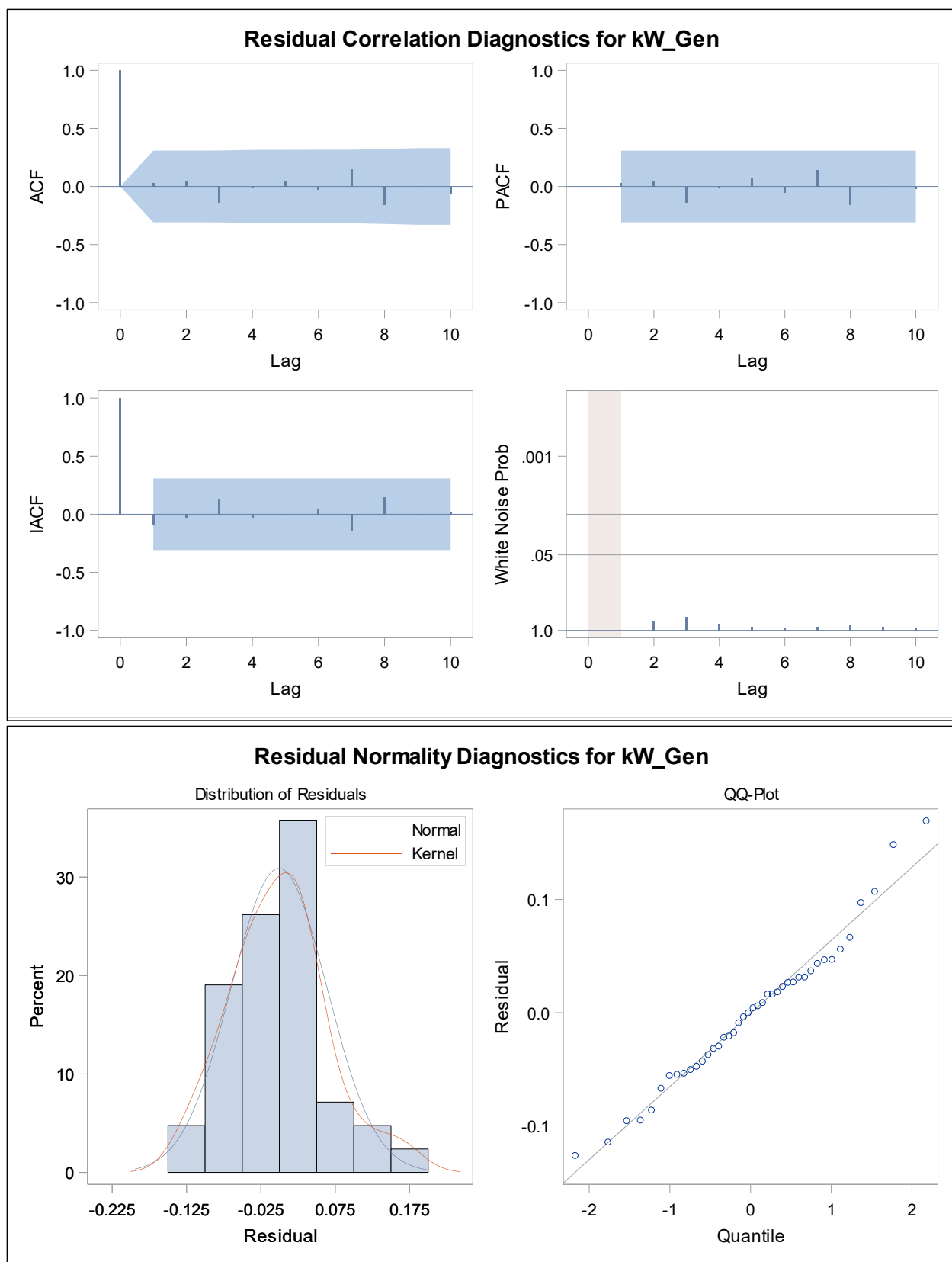
The AIC and SBC values (-104 and -97, respectively) are the best from all models that were run up to this point.

Constant Estimate	0.443121
Variance Estimate	0.0045
Std Error Estimate	0.067082
AIC	-103.598
SBC	-96.6469
Number of Residuals	42

Correlations of Parameter Estimates					
Variable Parameter	kW_Gen MU	kW_Gen AR1,1	cosval NUM1	Cloud_Cover NUM2	
kW_Gen MU	1.000	-0.036	-0.028		-0.919
kW_Gen AR1,1	-0.036	1.000	-0.034		0.037
cosval NUM1	-0.028	-0.034	1.000		0.065
Cloud_Cover NUM2	-0.919	0.037	0.065		1.000

Autocorrelation Check of Residuals									
To Lag	Chi-Square	DF	Pr > ChiSq	Autocorrelations					
6	1.24	5	0.9407	0.029	0.046	-0.139	-0.017	0.052	-0.028
12	7.87	11	0.7245	0.145	-0.160	0.006	-0.071	-0.199	-0.155
18	16.61	17	0.4814	-0.039	0.067	-0.064	-0.242	-0.219	0.069
24	19.11	23	0.6950	0.022	0.111	0.118	0.040	0.007	0.023

The residuals from this model also appear to be white noise.



Model for variable kW_Gen	
Estimated Intercept	0.997445

Autoregressive Factors	
Factor 1:	$1 - 0.55574 B^{**}(1)$

Input Number 1	
Input Variable	cosval
Overall Regression Factor	0.166638

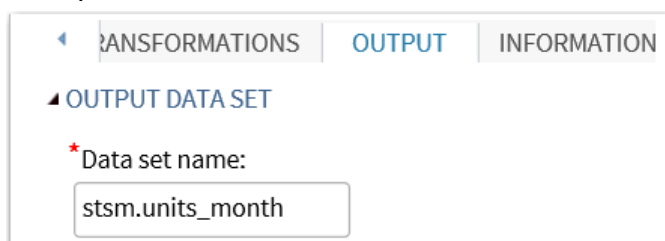
Input Number 2	
Input Variable	Cloud_Cover
Overall Regression Factor	-0.09091

End of Demonstration

Unobserved Components Model

Recall that you explored a monthly, intervalled time series that showed evidence of trend and seasonal component variation. This series was created from time-stamped observations on **units** in the **CH1_DEMODAT** table. The first part of this demonstration creates a new table, **stsm.units_month**, that aggregates the **units** time series to a monthly interval using the average aggregation method. A UCM model is then specified and fit to **units**.

1. Create a new Time Series Data Preparation task in SAS Studio.
2. On the DATA tab, set the data option to **STSM.CH1_DEMODAT**. Set the time series variable to **units**. Set the time ID to **date**, and set the interval of the time ID variable to **Month**.
3. Click the **TRANSFORMATIONS** tab, and set the Accumulation method to **Average**.
4. Click the **OUTPUT** tab, and change the output data set name to **units_month** in the **STSM** library.



The screenshot shows the SAS Studio interface with the **OUTPUT** tab selected. Under the **OUTPUT DATA SET** section, the **Data set name:** field is populated with **stsm.units_month**.

Note: Alternatively, write the SAS code directly as follows:

```
proc timeseries data=stsm.CH1_DEMODAT seasonality=12
                out=stsm.UNITS_MONTH;
    id date interval=month;
    var units / accumulate=average transform=none dif=0 sdif=0;
run;
```

5. Run the task.

End of Demonstration

Build a UCM and explore the results

1. Expand the **Forecasting** tasks, and create a new Modeling and Forecasting task.
2. On the DATA tab, assign table and variable roles as described: Specify **STSM.UNITS_MONTH** as the DATA table; **units** in the Dependent variable field; and **date** in the Time ID field.

The screenshot shows the 'DATA' tab of a forecasting task configuration window. The 'DATA' section has a dropdown menu set to 'STSM.UNITS_MONTH'. Below it, a 'NOTE' section states: 'This task requires data in a valid time series format. To prepare your data, run the Time Series Data Preparation task before starting this task.' The 'ROLES' section has a 'Dependent variable (1 item)' field with 'units' selected. The 'ADDITIONAL ROLES' section has a 'Time ID (1 item)' field with 'date' selected. At the bottom, the 'Interval' is set to 'Month'.

3. On the MODEL tab, select **Unobserved Components** as the forecasting model type.
4. Recall that the component analysis from Chapter 1 suggested that the monthly, intervalled **units** data has trend, seasonal, and irregular (ARMA type) patterns.
 - a. To accommodate these, modify the default model settings by selecting the **Include a slope component** check box.
 - b. Also, expand the **Seasonal Component** dialog box.
 - c. Select the **Include a seasonal component** check box.

These settings are summarized in the image below.

Note: The level and slope components combine to accommodate a trend in the model.

The screenshot shows the 'MODEL' tab of a forecasting software interface. At the top, there are four tabs: 'DATA', 'MODEL' (selected), 'OPTIONS', and 'OUT'. Below the tabs, the 'MODEL' section is expanded, showing a dropdown menu for 'Forecasting model type' set to 'Unobserved components'. Under 'Model Settings', there are three main sections: 'Irregular Component', 'Trend Component', and 'Seasonal Component'. Each section has a checkbox to 'Include' the component and a checkbox to 'Specify variance'. In the 'Seasonal Component' section, the 'Include a seasonal component' checkbox is checked, and the 'Type' dropdown is set to 'Dummy'.

Forecasting model type: Unobserved components

Model Settings

Regression Effects

Irregular Component

☒ Include an irregular component

☐ Specify variance

Trend Component

☒ Include a level component

☐ Specify variance

☐ Check for level breaks

☒ Include a slope component

☐ Specify variance

Seasonal Component

☒ Include a seasonal component

Type: Dummy

☐ Specify variance

5. Expand the **Plots** dialog box at the bottom of the MODEL tab, and select **Selected plots** in the Selected Plots to Display box.
6. In addition to the default plots, select the following:
 - **One Step Ahead Forecasts**
 - **Irregular Component**
 - **Season Component**
 - **Level Component**
 - **Slope Component**

Note: Alternatively, you can write the SAS code directly as follows:

```
/* Build and Explore a UCM */
proc ucm data=stsm.UNITS_MONTH;
  id date interval=month;
  model units;
  irregular plot=smooth;
  level plot=(smooth);
  slope plot=(smooth);
  season length=12 type=dummy plot=(smooth);
  estimate plot=(panel model loess);
  forecast lead=12 back=0 alpha=0.05 plot=(forecasts);
  outlier;
run;
```

Additional plot options were added to the component statements:

- The PLOT=(SMOOTH) options produce smoothed representations of the level, slope, and season components.
- The PLOT=(PANEL, MODEL, and LOESS) options in the ESTIMATE statement produce residuals diagnostics plots.
- The PLOT=(FORECASTS) option in the FORECAST statement produces a plot of historical and lead forecasted values.

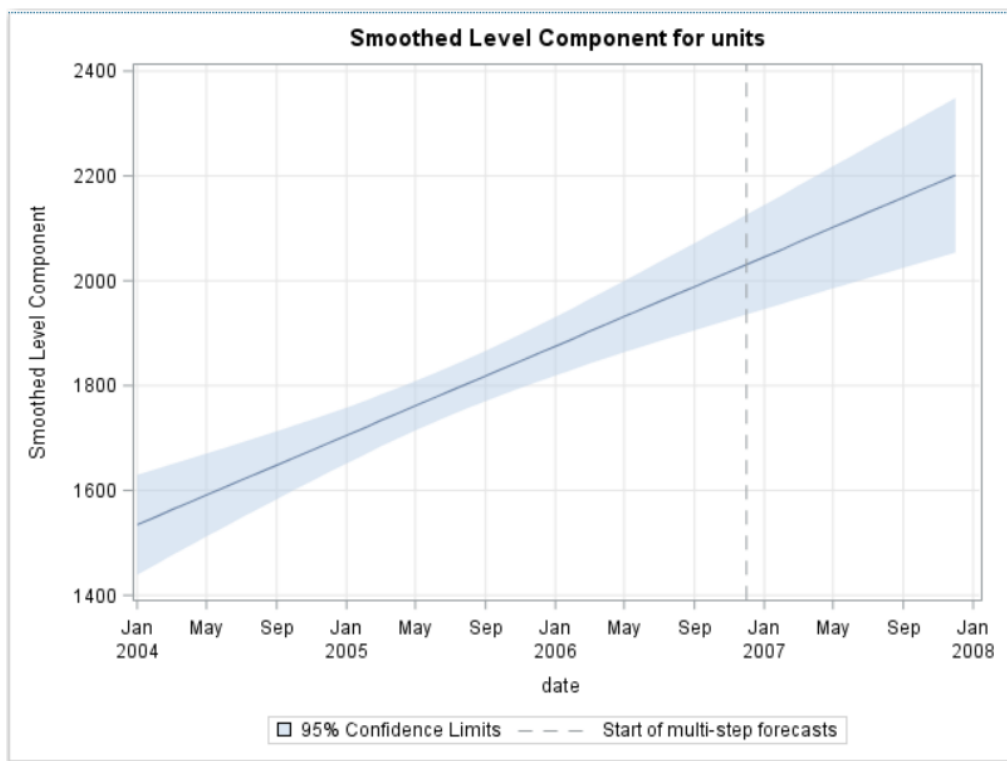
7. Select **Run** to submit the generated UCM syntax.

The Significance Analysis of Components table indicates that the data contain a significant level, slope, and seasonal component. However, the irregular component seems to be negligible in the presence of other components in the model. The slope estimate suggests that the data is increasing by approximately 14 units per month.

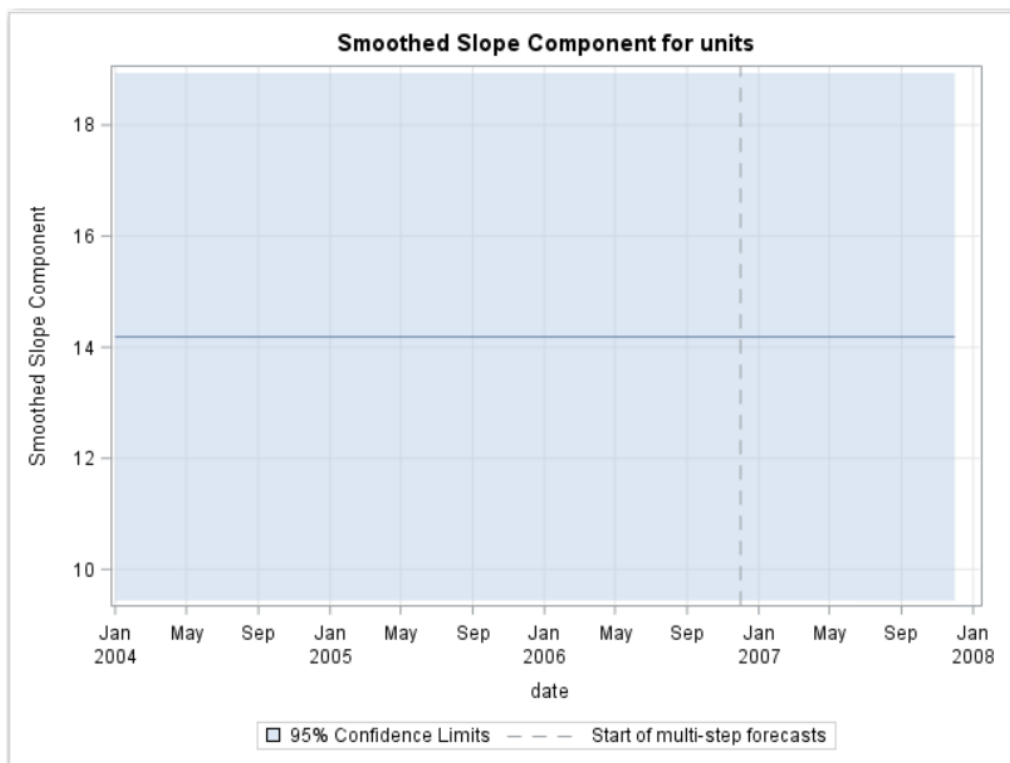
Significance Analysis of Components (Based on the Final State)			
Component	DF	Chi-Square	Pr > ChiSq
Irregular	1	0.00	0.9967
Level	1	1748.27	<.0001
Slope	1	34.28	<.0001
Season	11	26.55	0.0054

Trend Information (Based on the Final State)		
Name	Estimate	Standard Error
Level	2031.166251	48.578223
Slope	14.18644584	2.4228224

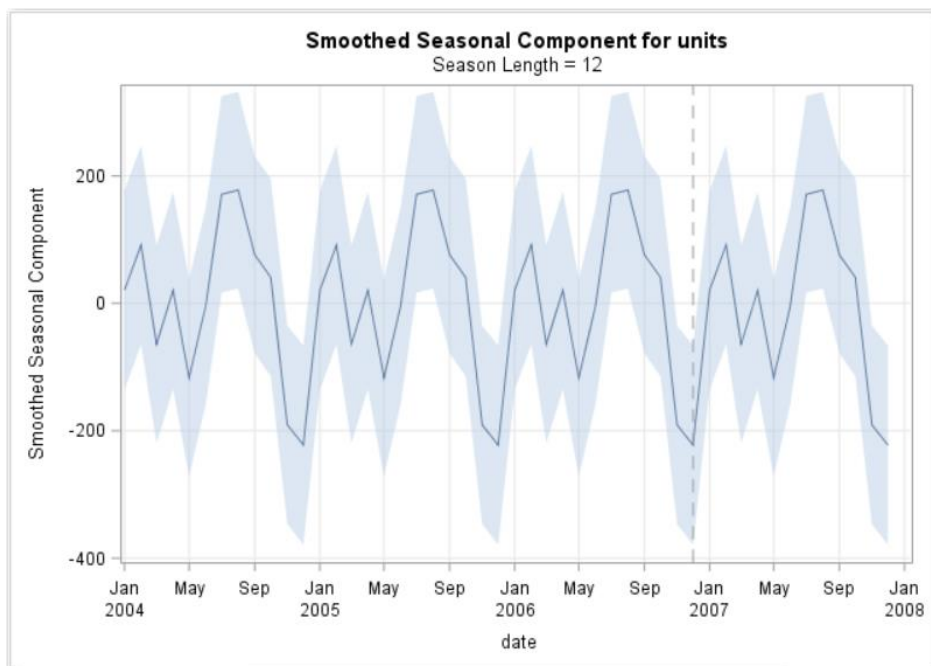
The Smoothed Level Component plot shows how the level of the data is estimated to evolve over the history and future forecast horizon.



The Smoothed Slope Component plot indicates that the slope in the data is a constant, and that the trend component is basically a deterministic linear trend.

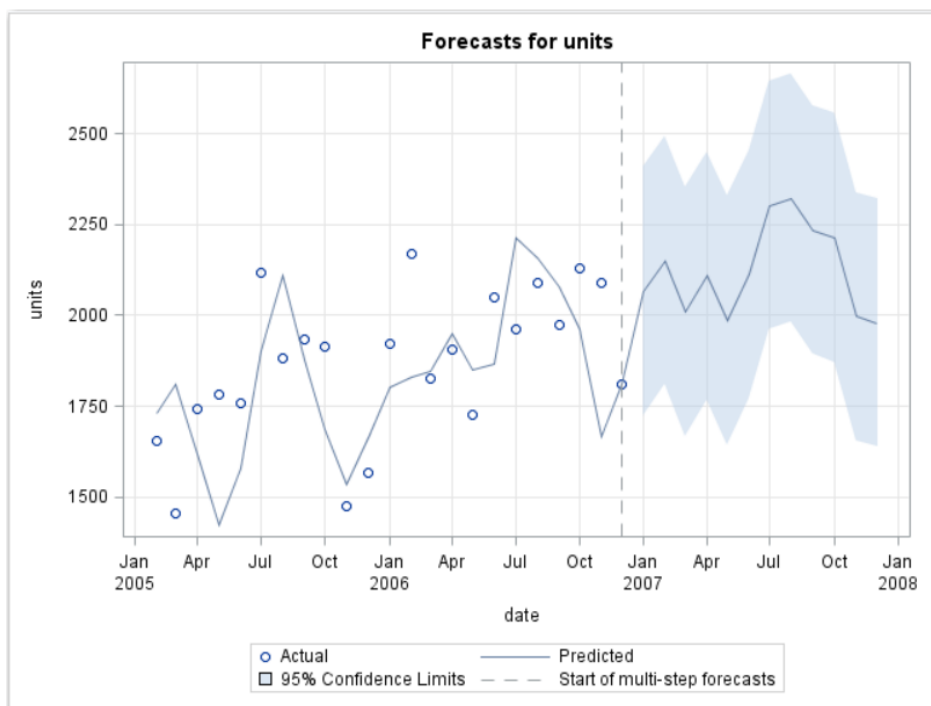


The Smoothed Seasonal Component plot shows the sample and lead forecasts from the seasonal component model.



Note: Smoothed component plots are based on parameter estimates that are derived from all the observations in the data.

The forecast plot shows the extrapolated trend, level, and seasonal components in the lead forecast. The forecast is the sum of forecasts from the estimated components that are listed in the Significance Analysis of Components table.



End of Demonstration

Time Series in SAS Viya Model Studio

Data Dictionary

You will work with a transactional data set named **lookingglass_forecast**. The data consist of information from a telecommunications group whose goal is to predict sales.

The data set is already accumulated into a time series with **Txn_Month** as the time variable. Potential predictor variables of the continuous response **sale** are **price**, **discount**, and **cost**. The data set also contains the attribute variables that will be used in hierarchical forecasting, **productline** and **productname**.

Variables in LOOKINGGLASS_FORECAST					
Variable	Type	Length	Format	Informat	Label
Txn_Month	Num	8	MMDDYY10.	DATE15.	Transaction Date (Month)
productline	Char	8			Name of product line
productname	Char	12			Product name
sale	Num	8			Unit Sale
price	Num	8			Unit Price
discount	Num	8			Price Discount
cost	Num	8			Unit Cost

Setting up a Visual Forecasting Project

In this section, you create a new forecasting project in Model Studio, **baseline sales forecasts**, and load data into the project. The **baseline sales forecasts** project is used throughout the workshop.

1. Navigate to the upper left corner in SAS Viya Landing Page, click **Applications > Build Models**. This takes you to Model Studio. Click on the **New Project** button.

Model Studio is an integrated visual environment that provides a suite of analytic tools to facilitate end-to-end data mining, text, and forecast analysis. The tools in Model Studio are designed to take advantage of SAS Viya programming and cloud processing environments to deliver and distribute the results of the analysis, such as champion models, score code, and results. It does all this fast.

Note: If this is your first session, there will be no existing projects unless projects were set up for you. (If projects already exist, the **New Project** button is available in the upper right corner.)

2. Name your project **baseline sales forecasts**.

Note: Naming your project something relevant and adding a reasonably detailed description of the project is considered a forecasting best practice.

3. For **Type**, select **Forecasting**.

There are three types: Data Mining and Machine Learning, Forecasting, and Text Analytics. This workshop will only deal with Forecasting.

Once you select a type and you create your project, you can't change the Forecasting type. So, you would have to reopen a new project. Make sure you've got the right type before you save the project at the bottom of this window.

4. For **Template**, use the drop-down menu or the Browse button and choose **Auto-forecasting**.
5. For **Data Source**, click **Browse** to select the modeling data source.
6. Make sure the Show area says **Available** to view in-memory tables that are available for model building. Scroll and find the **LOOKINGGLASS_FORECAST** in-memory table. Alternatively, you can click on the search bar and type lookingglass and click the magnifying glass icon or hit Enter.
7. Select **LOOKINGGLASS_FORECAST**, click **OK**, and then **Save** to create the new project. The project now appears.
8. Ensure that the project's **Data** tab is selected to assign variable roles.

A Note on Variable Assignment:

- Individual variables can be selected for role assignment by either clicking the variable name or by selecting the corresponding check box.

- Individual variables are **deselected** after their role is assigned by either **clearing their check box** or **selecting another variable's name**.
- More than one variable can be selected at the same time using the check boxes.
- Because selecting a variable using the check box does not deselect others, it is easy for new users to **inadvertently re-assign** variable roles. Taking a few minutes to get comfortable with the variable selection functionality is considered a best practice for using the software.

9. The **Txn_Month** variable is assigned to the role of Time for the project.

Note: Other time intervals are available by selecting the down arrow next to **Month**. The time interval combined with the Multiplier and Shift options indicates that the desired interval of the time series data is one month and that the 12-month annual cycle starts in January. These options can be changed to modify the time index if it is appropriate for your data.

10. **Sale** is the target for the analysis. Click **sale** in the middle variables list panel. In the right property panel, select **Dependent**.

Note: Missing interpretation options enable the user to interpret, or impute, values for embedded missing values in the series. By default, embedded missing values have no value assigned to them.

11. Deselect **sale**. Assign **productline** and **productname**, in that order, to the **BY Variable** role.

12. Set Reconciliation level to **productline**.

13. Additional variables will be assigned to roles. **Price**, **discount**, and **cost** can be useful as explanatory variables in subsequent analyses. Select these three variables, where the order does not matter, and change their roles to **Independent**.

Note: For each of these variables, accumulation is accomplished by averaging observed values in each month.

14. Change **Usage in system-generated models** to **Try to Use**.

If I select Force to use, then each of these three variables will be used in every one of the models for every one of the series. I don't want to do that. Let's see some other options-- Try to use and Use if significant.

Try to Use will test each of the variables in each of the series. So, for each model, for each of the 918 series, in the dataset each of the variables will be tested to see if they're statistically significant in the model, and whether they benefit the model with respect to a fit statistic, such as Akaike's information criterion. So, there are two criteria. If a variable passes both of those tests, they'll be used in the model. If the variable doesn't, it won't be used in the model.

Try to Use is slightly different from Use if significant. Use if significant only tests to see if the variable is statistically significant and doesn't check to see if it improves the model's fit statistic.

So I'm going to use Try to use. Now my data are ready to start my pipelines.

Creating Pipelines

In this section, you perform basic forecasting with a pipeline.

1. Navigate to the **Pipelines** tab.

The Auto-Forecasting template is the default pipeline template for Visual Forecasting. It consists of the essential steps in a forecasting analysis:

- accumulates the data into time series
- automatically identifies, estimates, and selects forecast models for the time series
- assesses forecasting results
- publishes results for use outside the pipeline

Note: If the modeling data are hierarchically arranged, the identification, estimation, and selection steps in the default forecasting pipeline are done on series in the base level of the hierarchy.

2. Select **Run Pipeline** in the upper right corner of the workspace.

Then we see the circles, the empty circles start up. And by the time we see all circles filled with green ink and a white checkmark, then the pipeline will be completed. Now that it's done, let's look at the Auto Forecasting node.

Auto-forecasting Node Results

1. Right-click on the **Auto-forecasting** node and select **Results**.

Because the Auto-forecasting node is designed to be run with minimal input from the analyst, relatively few options are surfaced for this node. The Auto-forecasting node automatically identifies, estimates, and generates forecasts for the 918 series in the base or product name level of the modeling hierarchy. Most of the forecast models selected for these series are in the ARIMAX family.

For each series, three families of time series models are considered by default: ARIMAX (ARIMA with exogenous variables), ESM (exponential smoothing models), and IDM (intermittent demand models). The champion model for each series is chosen based on root mean square error. Other selection statistics are available in the Model selection criterion option.

The MAPE Distribution histogram is in the upper left-hand corner. The distribution of Mean Absolute Percent Error (MAPE) for forecasts in the product name level of the hierarchy can be used to compare the accuracy of different forecast models. Each of the bars represents the proportion of the series that have a specific range of MAPE values. In general, smaller values of MAPE imply greater accuracy.

The Model Family histogram is in the upper right. Among each of these 918 series, we can see what percentage of each model family was selected as the best across all the time series.

The Model Type chart, located in the lower left, summarizes systematic variation found in the identification process. This chart shows information about the selected models across all the time series.

What percentage of the models has a trend component, a seasonal component, or an predictor variable used? They can be overlapping series; therefore, we can see that the percentage is summed to over 100%.

The Execution Summary, located in the lower right, provides information about results that are potentially problematic, anomalous, or both. We can see there were 918 series. There weren't any series that failed for forecasting. There were only six series with forecasts equal to zero, meaning in the forecast range, the forecasts were all zero. Then there is a lot of summary information about the number of series that had flat forecasts. Flat forecasts means that in the forecast range, the forecast values were a constant.

2. Click the **Output Data** tab above the MAPE Distribution plot.

Several output tables are created. You can view them by clicking on them.

You'll notice that there is a unique product line, product name combination for each line. So that's a unique series identified by its product line and product name. And notice that we have multiple lines of data in this data set, each for a different month. So, if I scroll down, I can see that the Time ID. And once we get to 2017, we see the actual values is missing. What that means is that this is the forecast horizon. The forecast horizon, of course, doesn't have any actual values, but it can have predicted values and so on. So that's information you might want to obtain from that forecast table.

3. Click the **OUTMODELINFO** data source to open it.

For each series, the selected model is named, and attributes of the model are displayed.

Once again, we see information for every product line, product name combination. In other words, every series. And in this data set, we see the name of the model, or the label for the model, is the type of model that was chosen as the champion model for that series. So, for Line01, Product01, that series, it was an ARIMA model with regression parameters. It is under the ARIMA family. There were no dependent variables here. And we can get information about whether there are seasonal components, whether there are trend components, whether there are inputs presence, and so on.

So that's information for the Champion model. If I want to see what the competitors were, we can click on the OUTSELECT Data Source. And now you'll see there are three lines for each one of the series. So Line01, Product01 is three different lines. And you can see which of the models were under consideration by looking at the Model column. And then the next column for Selected Status, you can see that the selected row, the selected model for this series, as we'd seen before, was the ARIMA model with regression parameters. And you can see why that happened by scrolling farther to the right, each one of these has fit statistics and accuracy statistics calculated. So, if we looked at the Mean Absolute Percent Error column, for those first three rows, you could see why that model won the competition for that series.

4. Close the **Results** window.
5. Right-click and open the results of the **Model Comparison** node.

The Champion Model is the Auto-forecasting model, which is the only one included in the pipeline. WMAE and WMAPE are weighted sums of the MAE and MAPE values across all series. WMAPE and WMAE represent average performance of all the models in a modeling node.

Note: For the WMAPE and WMAE, the final computation is based on weighted measurements from each time series, where more weight is given to time series with a higher average of the dependent variable.

6. Close the **Results** window.

Hierarchical Modeling Node

Creating a Pipeline, Adding, and Running the Hierarchical Modeling Node

1. Select the **Pipelines** tab, and then select **Add new pipeline (+)**.
2. Name the new pipeline **Hierarchical Modeling**.
3. Next to template, select the **Browse** button, and then expand **Base Forecasting**. This will create a pipeline with just a Data node. We will be adding the Hierarchical Modeling node to this pipeline.
4. Select **OK**, and then select **Save**.
5. On the left pane, select the **Nodes** menu, and then expand **Forecasting Modeling**.
6. Left click, drag and drop a **Hierarchical Modeling node** on top of the **Data** node.
7. Run the pipeline.

Exploring the Hierarchical Modeling Node

1. When the run completes, right click the **Hierarchical Modeling** node and select **Open**.

The three levels in the data hierarchy are listed in the table. Additionally, a note reports that the Reconciliation process was completed. Weighted MAPE, denoted WMAPE, and Reconciled WMAPE values are listed for each level of the hierarchy.

Note: WMAPE is a measure that aggregates MAPE values associated with the champion model's forecast for each series in the corresponding level of the hierarchy. Reconciled WMAPE is a measure that aggregates MAPE values on the reconciled forecasts in the corresponding level of the hierarchy.

2. Check the box next to the **TOP** level, and then click the **Open Pipeline** button at the top right.
3. Right-click on the Auto-forecasting Node in the pipeline. Choose **Forecast Viewer**.
4. Select **Top** in the right-hand-panel. This will activate the plots on the page.

The plot shows the actuals and forecasts associated with the data at the top level of the data hierarchy. Top level **SALES** seem to be trending up nicely. Also, the total **SALES** series has a seasonal pattern that the champion model captures and extrapolates into the lead forecast horizon.

Note: The TOP level data is an aggregation of Unit Sales (SALE) across all BY groups in the project's data. It is a single series that results from the process of aggregation and represents total unit sales across all **productname** or **productline** series.

5. Close Forecast Viewer and the **Top** pipeline to return to the open area of the Hierarchical Modeling node. Check the box next to the **productline** level. Click the **Open Pipeline** button at the top right.

6. Right-click on the Auto-forecasting Node in the pipeline. Choose **Forecast Viewer**.
7. An envelope plot showing information about all 270 **productline** series will appear. Selecting a series in the right-hand-panel will emphasize that series in the plot.

This takes you to the **Forecast Viewer** for the pipeline associated with the middle of the project's data hierarchy. There are 270 **productline** series in the middle level of the data hierarchy.

8. Close the **Forecast Viewer** and the **productline** pipeline to return to the open area of the Hierarchical Modeling node. Check the box next to the **productname** level. Click the **Open Pipeline** button at the top right.
9. Right-click on the Auto-forecasting Node in the pipeline. Choose **Forecast Viewer**.
10. An envelope plot showing information about all 918 **productname** series will appear. Selecting a series in the right-hand-panel will emphasize that series in the plot.

This takes you to the **Forecast Viewer** for the pipeline associated with the bottom level of the project's data hierarchy. There are 918 **productname** series in the middle level of the data hierarchy.

11. Close the **Forecast Viewer** and the **productname** pipeline to return to the open area of the Hierarchical Modeling node.

Exporting the Reconciled Forecasts for the Project

The three Forecast Viewers that we just explored are associated with three different pipelines, one for each level of the data hierarchy. Each pipeline is generated using an **AUTO-FORECASTING** template.

Recall that when this project was created, the Reconciliation Level was set to **productline**. When the **Hierarchical Modeling** node was run, automatic model generation, model selection and forecast generation were accomplished for all three levels of the data hierarchy. As a final step, the forecasts at the **Top** and **productname** levels were reconciled to the forecasts at the **productline** level. Now, we're going to export the generated reconciled forecasts for this project.

Saving the Forecast Data from the *productline* Pipeline

1. Select the check box next to the **productline** level in the **Hierarchy Levels** table.
2. Select the **Open Pipeline** button on the top right.
3. Inside the productline Pipeline, select the **Nodes** button and then expand **Miscellaneous**.
4. Left click, drag, and drop a **Save Data** node on top of the **Auto-forecasting** node.
5. Select the **Edit save options** button in the properties of the **Save Data** node.
6. Expand **Time Series** under **Output Tables** and select **Forecasted values**.
7. Check the box next to **Include in output**.
8. Rename the table **productline_OUTFOR**.

Note: Because we've set reconciliation to the middle, or productline, level, the productline level forecasts are not impacted by the process of reconciliation; productline level statistical forecasts are the Predicted Values (PREDICT) in the OUTFOR table from this level of the hierarchy.

9. Select the Browse button to select an output CAS library.
10. Expand **cas-shared-default** and select the **Public** library.
11. Select **OK**, and then select **OK** again to close the save options.
12. Close the **productline Pipeline**.

Saving the Forecast Data from the *productname* Pipeline

1. Select the check box next to the **productname** level in the **Hierarchy Levels** table.
2. Select the **Open Pipeline** button on the top right.
3. Inside the Productline Pipeline, select the **Nodes** button and then expand **Miscellaneous**.
4. Left click, drag, and drop a **Save Data** node on top of the **Auto-forecasting** node.
5. Select the **Edit save options** button in the properties of the Save Data node.
6. Expand **Time Series** under **Output Tables** and select **Forecasted values**.
7. Check the box next to **Include in output**.
8. Rename the table **productname_RECFOR**.

Note: Because we've set reconciliation to the middle, or **productline**, level, the **productname** level forecasts are impacted by the process of reconciliation. **Productname** level reconciled forecasts are the **Predicted Values (PREDICT)** in the **OUTFOR** table from this level of the hierarchy.

9. Select the Browse button to select an output CAS library.
10. Expand **cas-shared-default** and select the **Public** library.
11. Select **OK**, and then select **OK** again to close the save options.
12. Close the **productname Pipeline**.

Exporting the Reconciled Forecasts from the Project

1. Close the **Hierarchical Modeling** node.
2. **Run** the Hierarchical Modeling Pipeline.

Tables exported by the Save Data node are promoted by default.

3. Select the **Applications** menu and then select **Manage Data**.
4. In left pane, expand **cas-shared-default** and choose **Public**.
5. Next to the filter indicate that we would like in-memory data to be shown only by selecting the button to active.
6. Within the list you will now see the new output data sets that we created with the **Save Data Nodes**.

The three in-memory reconciled forecast tables we exported from our project are available here for further processing and reporting.